High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs

A thesis submitted

in partial fulfillment for the award of the degree of

Master of Technology

in

Machine Learning and Computing

by

Adarsh Dubey



Department of Mathematics

Indian Institute of Space Science and Technology

Thiruvananthapuram, India

May 14, 2026

Certificate

This is to certify that the thesis titled ***High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs*** submitted by **Adarsh Dubey**, to the Indian Institute of Space Science and Technology, Thiruvananthapuram, in partial fulfillment for the award of the degree of **Master of Technology** in **Machine Learning and Computing** is a bonafide record of the original work carried out by him/her under my supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Dr. S. Sumitra
Professor, Department of **Mathematics**
IIST, Thiruvananthapuram

Dr. Deepak T G
H.O.D. of **Mathematics**
IIST, Thiruvananthapuram

Place: Thiruvananthapuram
Date: May 14, 2026

Declaration

I declare that this thesis titled ***High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs*** submitted in partial fulfillment for the award of the degree of **Master of Technology** in **Machine Learning and Computing** is a record of the original work carried out by me under the supervision of **Dr. S. Sumitra**, and has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other titles in this or any other Institution or University of higher learning. In keeping with the ethical practice in reporting scientific information, due acknowledgments have been made wherever the findings of others have been cited.

Place: Thiruvananthapuram
Date: May 14, 2026

Adarsh Dubey
(SC24M151)

Dedicated to my family and teachers.

Acknowledgements

I express my sincere gratitude to my academic supervisor, **Dr. S. Sumitra**, Professor, Department of Mathematics, Indian Institute of Space Science and Technology, for her continuous guidance, encouragement and invaluable feedback throughout the course of this work.

I am equally grateful to my industry supervisor, **Mr. Manoj Kumar**, Manager, Intel Corporation, for hosting this project, for providing access to the required compute infrastructure, and for mentoring me on the engineering aspects of large-scale deep-learning systems.

I thank the faculty and staff of the Department of Mathematics at IIST for their support, and my colleagues at Intel for the many technical discussions that shaped this work. Finally, I am deeply thankful to my family and friends for their unwavering support.

Adarsh Dubey

Abstract

Training and serving a modern Large Language Model stresses every layer of the deep-learning software stack at once: the distributed schedule must keep thousands of accelerators coordinated, the kernel layer must drive Tensor Cores near peak, the attention algorithm must scale sub-quadratically with the context window, and the serving runtime must turn the accumulated algorithmic savings into end-user throughput. Isolated optimisations at any one of these layers leave the gains of the others on the table. This thesis develops, implements, and evaluates an end-to-end stack in which the five layers are designed to compose, and measures its behaviour on NVIDIA A100 hardware across both training and inference workloads.

The work addresses LLM optimisation along five complementary axes. *First*, a systematic comparative study of four distributed training strategies—Data Parallelism (DP), Distributed Data Parallel (DDP), Fully Sharded Data Parallel (FSDP), and DeepSpeed ZeRO-3—is carried out on a single-node 8-GPU A100 server with identical workloads, identical initialisations, and identical random seeds, isolating the effect of the parallelisation strategy itself. *Second*, low-level matrix-multiplication kernels are built using NVIDIA CUTLASS 3.x and the CuTe layout algebra, covering threadblock/warp/instruction tiling, multi-stage software pipelining, Tensor-Core utilisation, and epilogue fusion for bias, activation, and residual additions. *Third*, a novel block-sparse *Sliding Window Flash Attention* (SWFA) algorithm is developed that augments Flash Attention with an explicit block-classification step and a vectorised mask generator. Entire $B_r \times B_c$ blocks that fall outside the attention band are skipped rather than merely masked. *Fourth*, GPU kernels inside the SGLang serving runtime are engineered for long-context inference, including paged KV cache access, fused decode attention, and fused top-k/top-p sampling. *Fifth*, a Triton-based kernel profiling and auto-tuning pipeline is assembled that combines CUDA-event timing, roofline analysis, Nsight Compute cross-checks, and shape-indexed configuration caching.

Experimental results on an 8-GPU A100-40GB server show that DeepSpeed ZeRO-3 attains the highest GPU utilisation (93.8% peak, 65.8% average) and the best memory efficiency ($\approx 87.5\%$ reduction per GPU over DP), while FSDP provides the best memory/throughput trade-off for medium-scale models. The proposed SWFA delivers up to 65.6% reduction in computed attention blocks for narrow windows while matching the

reference output within 10^{-4} FP32 tolerance and 10^{-2} FP16 tolerance across nineteen validation configurations. When combined with DeepSpeed or FSDP on a GPT-2-Medium workload with sequence length 2048, SWFA enables end-to-end training throughput improvements of 50–59% and energy savings of 25–29% per epoch. A closed-form expression for the optimal window size under an exponential attention-decay cost model is derived, and strict convexity of the corresponding loss is proved. The SGLang kernel layer achieves a cumulative $2.84\times$ decode-throughput speedup on LLaMA-7B, and the Triton profiling pipeline reproduces these numbers independently with $<3\%$ deviation.

Keywords: Large Language Models, Distributed Training, Flash Attention, Sliding-Window Attention, CUTLASS, CuTe, SGLang, Triton, GPU Computing, Kernel Profiling, Tensor Cores.

Contents

List of Figures **xv**

List of Tables **xvii**

List of Algorithms **xix**

Abbreviations **xxi**

Nomenclature **xxiii**

1 Introduction **1**

1.1 Background and Motivation 1
1.2 Problem Statement 3
1.3 Objectives 4
1.4 Contributions 5
1.5 Scope of the Work 6
1.6 Methodology Overview 7
1.7 Timeline 7
1.8 Thesis Organisation 8

2 Literature Review **11**

2.1 Distributed Training of Deep Neural Networks 11
2.2 High-Performance GPU Kernels and GEMM Libraries 14
2.3 Efficient Attention Mechanisms 15
2.4 LLM Serving Systems and KV-Cache Management 18
2.5 GPU Kernel Profiling and Auto-tuning 18
2.6 Energy and Carbon Considerations 19
2.7 Cross-Cutting Concerns 19

2.8 Positioning of This Thesis 20

3 Distributed Computing and Training **23**
3.1 Background 23
3.2 Methodology 26
3.3 Implementations 28
3.4 Results 30
3.5 Discussion and Recommendations 33
3.6 Summary 34

4 CUTLASS and CuTe GEMM Operations **35**
4.1 Motivation 35
4.2 The CUTLASS Programming Model 36
4.3 Software Pipelining 37
4.4 CuTe Layouts 38
4.5 Mapping LLM Layers onto GEMM 39
4.6 Epilogue Fusion 40
4.7 Methodology 41
4.8 Results 42
4.9 Summary 43

5 Sliding Window Flash Attention **44**
5.1 Motivation 44
5.2 Background 45
5.3 Method 46
5.4 Correctness and Performance 51
5.5 Optimal Window Size 54
5.6 Summary 56

6 GPU Kernels in the SGLang Runtime **57**
6.1 Background: SGLang 57
6.2 Execution Model 59
6.3 Kernel-Level Optimisations 60
6.4 Public Baselines from the Prior Art 62
6.5 Methodology 62
6.6 Results 63

6.7 Discussion . . . 65
6.8 Summary . . . 66

7 Triton Kernel Profiling and Auto-tuning 67
7.1 Triton Programming Model . . . 67
7.2 Methodology . . . 68
7.3 Implementation . . . 69
7.4 Results . . . 70
7.5 Triton Implementation of SWFA . . . 72
7.6 Discussion . . . 73
7.7 Summary . . . 73

8 Conclusions and Future Work 75
8.1 Summary of Contributions . . . 75
8.2 Lessons Learned . . . 76
8.3 Limitations . . . 77
8.4 Future Work . . . 77
8.5 Closing Remarks . . . 79

Bibliography 79

List of Publications 85

Appendices 87

A Algorithms and Reference Implementations 87
A.1 Block Classification . . . 87
A.2 Vectorised Mask Generation . . . 89
A.3 Full SWFA Forward Pass . . . 91
A.4 Full SWFA Backward Pass . . . 94
A.5 Distributed-Training Reference Scripts . . . 95
A.6 CUTLASS Kernel Snippets . . . 96
A.7 Triton SWFA Kernel . . . 97
A.8 Numerical Validation Harness . . . 98

B Mathematical Proofs 100
B.1 Notation and Conventions . . . 100

B.2 Correctness of Corner-Based Block Classification 100
B.3 Block-Count Lemma 101
B.4 Convexity of the SWFA Cost 102
B.5 Closed-Form Optimal Window 102
B.6 Flash Attention Softmax Correctness 103
B.7 Communication-Cost Bounds for Distributed Strategies 105
B.8 Exponential Decay Assumption 106
B.9 Reproducibility of the Numerical Accuracy Bound 106

List of Figures

1.1 The proposed LLM software stack. Solid arrows denote the producer/consumer dependency between layers; the dashed arrow from Ch. 7 back to Ch. 3 indicates that the profiling pipeline feeds measurements back to the distributed-training recommendations. 8

3.1 Data parallelism on a three-GPU ring. Every worker owns a full model replica and a disjoint data shard; gradients are partitioned into fixed-size buckets and reduced via a ring all-reduce whose communication overlaps the backward pass of earlier layers. 24
3.2 GPU utilisation across distributed training strategies. 30
3.3 Training-loss convergence across strategies. 31

4.1 CUTLASS three-level tile hierarchy on Ampere/Hopper. A threadblock tile lives in shared memory, is partitioned into four warp tiles, and each warp tile is covered by a grid of instruction-level MMA fragments that issue directly to the Tensor Cores. 36

5.1 SWFA block classification on an 8×8 grid with window $w_l = w_r = 2$. Green cells ($\mathcal{I}$) require no masking; hatched orange cells ($\mathcal{X}$) receive the vectorised mask of Algorithm 5.2; grey cells ($\mathcal{O}$) are skipped entirely. The skipped fraction grows quadratically with N/w, yielding the $1 - 2w/N$ asymptotic ratio proved in Appendix B.3. 47
5.2 SWFA speed-up vs. sequence length at fixed window w=512. 54

6.1 RadixAttention prefix tree. The system prompt and the few-shot examples are stored once at interior nodes and referenced by every active request; only the request-specific suffix is duplicated. For chat-style workloads this cuts KV-cache memory by $\rho_{\text{shared}} \in [0.4, 0.8]$. 58

6.2 Paged KV cache layout. Each request owns a block table that maps logical token-block indices (0, 1, 2, . . .) into physical page identifiers drawn from a shared page pool. Because page sizes are fixed at 16 tokens, the allocator never fragments; two requests can interleave in the pool without touching each other's pages, and the attention kernel dereferences physical addresses through a single 32-bit indirection. 59

7.1 Measured kernels on the A100 roofline. 71

List of Tables

2.1 Principal prior systems organised by stack layer. 22

3.1 Per-step communication volume on a ring all-reduce topology. 26
3.2 Per-GPU memory usage (GB), ResNet-50 + CIFAR-10, 8 A100. 31
3.3 Power and energy per epoch. 32
3.4 Summary of scalability metrics. 32
3.5 Compute/communication overlap per training step. 33

4.1 HBM traffic with and without epilogue fusion, GPT-2 Medium MLP at $N{=}2048$, batch 4. 41
4.2 Representative GEMM performance for LLM shapes. 42
4.3 Tensor-Core pipeline utilisation for the CUTLASS configurations of Table 4.2. 43

5.1 Numerical accuracy of SWFA on representative configurations. 52
5.2 Block-skip ratio as a function of window configuration. 52
5.3 Effect of block size on SWFA ($N{=}4096$, $w{=}512$). 53
5.4 End-to-end training throughput and energy with SWFA. 53
5.5 Optimal window $w^\star$ for representative parameter values. 56

6.1 Public baselines from the systems literature, cited for context. All numbers are from the original papers on their stated hardware; only the relative improvement over an unpaged/vanilla baseline is comparable across papers. 62
6.2 Decode throughput with SGLang kernel optimisations. 64
6.3 Per-token decode latency (ms) at various batch sizes, LLaMA-7B with all five optimisations enabled. 64

7.1 Triton vs. CUTLASS vs. Torch-inductor on representative LLM kernels (TFLOPS, FP16, A100-40GB). 71
7.2 Triton auto-tune winners for representative GEMM shapes. 72

List of Algorithms

5.1 Block classification for the sliding window. 48
5.2 Efficient mask generation for an intersecting block. 49
5.3 Sliding Window Flash Attention (forward pass). 50
5.4 Sliding Window Flash Attention (backward pass). 51

A.1 CLASSIFYBLOCKS: detailed reference. 88
A.2 GENERATEMASK: reference version. 89
A.3 SWFA-FORWARD: full expanded reference. 91
A.4 SWFA-BACKWARD: full expanded reference. 94

Abbreviations

LLM	Large Language Model
DL	Deep Learning
DP	Data Parallelism
DDP	Distributed Data Parallel
FSDP	Fully Sharded Data Parallel
ZeRO	Zero Redundancy Optimizer
MoE	Mixture of Experts
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit
HBM	High Bandwidth Memory
SRAM	Static Random Access Memory
FLOP	Floating-Point Operation
GEMM	General Matrix-Matrix Multiplication
SIMT	Single-Instruction Multiple-Thread
CUDA	Compute Unified Device Architecture
SYCL	Single-source heterogeneous C++ (Khronos)
MLIR	Multi-Level Intermediate Representation
NCCL	NVIDIA Collective Communications Library
FA	Flash Attention
SWFA	Sliding Window Flash Attention
SDPA	Scaled Dot-Product Attention
MMA	Matrix-Multiply-Accumulate (Tensor-Core instruction)
WGMMA	Warp-Group Matrix-Multiply-Accumulate (Hopper)
TMA	Tensor Memory Accelerator (Hopper)
RoPE	Rotary Position Embedding
MQA	Multi-Query Attention
GQA	Grouped-Query Attention

SGLang	Structured Generation Language (serving runtime)
KV	Key–Value (cache)
AMP	Automatic Mixed Precision
FP16/FP32	16-/32-bit Floating Point
BF16	Brain Floating Point 16
AdamW	Adam with decoupled Weight decay

Nomenclature

N	Sequence length (in tokens/positions)
d	Embedding/head dimension
H	Number of attention heads
B	Batch size
B_r, B_c	Flash-Attention block sizes along query/key axes
T_r, T_c	Number of row/column blocks, $T_r = \lceil N/B_r \rceil$
w_l, w_r	Left/right sliding-window sizes
w	Total window size, $w = w_l + w_r + 1$
Q, K, V	Query, key and value matrices
O	Attention output
m_i, ℓ_i	Online softmax running max and sum for row i
P	Number of parallel processes/GPUs
κ	Per-FLOP hardware cost (seconds/FLOP)
λ	Attention decay rate (1/position)
A_0	Base attention importance
α, β, γ	Computational, memory and performance cost weights
$\mathcal{I}, \mathcal{X}, \mathcal{O}$	Fully-inside, intersecting and fully-outside block sets

Chapter 1

Introduction

1.1 Background and Motivation

Within only a few years, Large Language Models—all of them descendants of the transformer design [1]—have shifted from an interesting research artefact to the workload that dominates most newly installed supercomputing clusters. When one traces the path from the 175-billion parameters of GPT-3 [2], through the 540-billion of PaLM [3], and into the open-weight LLaMA line [4, 5], each new model is not so much a qualitative leap as a direct reading of the compute-loss scaling relationships [6, 7]: whatever the compute budget, the model that best minimises the loss turns out to be bigger, and fed more tokens, than the previous generation of folklore had assumed. From an engineering point of view, the implication is painful: every generation tightens the per-device memory budget, raises the communication volume demanded of the cluster, and stretches the context lengths on which attention has to be computed [8, 9]. A stack that sufficed for the previous generation almost never suffices for the next one without being retuned from top to bottom.

1.1.1 A Concrete Memory Accounting

A short back-of-the-envelope calculation is enough to make the constraints concrete. Consider a 7-billion-parameter transformer trained in FP16 mixed precision with Adam. Each parameter then costs $2\,\mathrm{B}$ for the weight itself, another $2\,\mathrm{B}$ for its gradient, and $4 \cdot 2\,\mathrm{B}$ more for the FP32 master copy together with the two moment buffers that Adam keeps—$14\,\mathrm{B}$ per parameter in total, or about $98\,\mathrm{GB}$ merely for the persistent model state. That figure already exceeds the $40\,\mathrm{GB}$ HBM of a single A100, and it ignores activations, which at a sequence of $N{=}2048$ and a batch of $B{=}4$ contribute roughly a further $20\,\mathrm{GB}$ per transformer layer. Scaling to a 70-billion-parameter model enlarges the state budget by a factor

of ten and the activation budget by about three—both d_{model} and the depth climb. Memory, in other words, is the first-order constraint on *what can be trained at all*, and it is precisely the constraint that determines every sharding decision made in this thesis.

1.1.2 The Four Challenges That Motivate This Thesis

Today's LLMs demand an amount of hardware that no single GPU can supply. Systems such as PaLM or LLaMA-3 cannot be trained at all without spreading the job across hundreds, sometimes thousands, of accelerators. The moment one steps beyond a single device, four mutually coupled difficulties appear at once:

- **Memory management.** Even a 7-billion parameter model, once its weights, gradients, and optimiser state are counted, overruns the $40\,\text{GB}$ HBM available on one A100. Adam-style updates [10] attach two extra tensors—the first and second moment estimates—to every parameter, effectively tripling the parameter storage, and mixed-precision recipes [11] additionally demand a separate FP32 master copy.

- **Communication overhead.** The cost of synchronising gradients across a data-parallel group is a function of the model's size rather than the batch's; unless communication is deliberately overlapped with compute, the accelerators sit idle waiting on NVLink or InfiniBand for most of the step.

- **Hardware heterogeneity.** A production stack is expected to run on a heterogeneous fleet—A100, H100, Intel Xe, AMD CDNA—each with its own memory hierarchy, its own Tensor-Core or matrix-engine ISA, and its own interconnect topology, so that the same training job can behave very differently depending on the SKU it happens to land on.

- **Attention efficiency.** Self-attention contributes $\mathcal{O}(N^2)$ work and, at long context, dominates the overall FLOP count. Masking an element-wise sparse pattern after the fact is not enough: the underlying dense GEMM is still dispatched over blocks whose outputs are later discarded.

On top of all this sits the energy and carbon footprint of training [12]. The accounting here is almost linear: a 10% gain in compute utilisation translates more or less directly into a 10% saving in electricity and embodied carbon. Any optimisation that cuts wall-clock time and energy per epoch simultaneously therefore has value that extends well beyond its raw speedup.

1.1.3 A Stack View of the Problem

In what follows, LLM efficiency is never treated as a single algorithmic problem. It is treated as a *stack* of interdependent layers, each of which has to be tuned with one eye on the layer above it and the other on the layer below:

1. **Distributed schedule** (Chapter 3). Decides which parts of the model, gradients, and optimiser state live on which GPU, and which collective operations keep them synchronised. Sets the memory envelope for every kernel call below it.

2. **Kernel layer** (Chapter 4). Provides the dense GEMMs, grouped GEMMs, and epilogue-fused primitives on which every transformer layer is built. Its Tensor-Core utilisation is inherited at every scale of parallelism.

3. **Attention algorithm** (Chapter 5). Determines the asymptotic complexity of the dominant operator at long context. A kernel that ignores the sparsity pattern exposed by the model architecture wastes the savings the upper layers have worked to create.

4. **Serving runtime** (Chapter 6). Turns algorithmic savings into end-user-visible throughput by scheduling kernel launches, managing the KV cache, and batching requests continuously.

5. **Profiling and auto-tuning** (Chapter 7). Closes the loop: it measures whether the four layers above are actually delivering their expected performance, and it selects per-shape kernel parameters that would otherwise have to be hand-tuned.

Two consequences of this perspective reappear in every chapter. First, no point optimisation at one layer is more useful than the next layer up permits it to be—a kernel speedup is wasted if a slow collective in the distributed schedule serialises everything behind it. Second, a measurement at one layer is only as reliable as the independent check performed one layer below it; a throughput figure at the top can easily conceal a regression in a kernel that a lower-level profile would have caught immediately. For these reasons the profiling pipeline described in Chapter 7 is treated as a first-class component of the contribution, not as a convenient side tool.

1.2 Problem Statement

The questions this thesis sets out to answer are the following.

1. **Scalable distributed training.** The comparisons of DP, DDP, FSDP, and DeepSpeed ZeRO-3 that one finds in the published literature are, almost without exception, produced by the authors of the individual system, usually on a different workload and a different hardware configuration each time. What the literature does not contain is a controlled single-node study in which the model, the data, the seed, and the hardware are all held fixed and only the parallelisation strategy is varied.

2. **High-performance matrix multiplication.** The GEMM kernels in vendor libraries such as cuBLAS are tuned aggressively for large, square matrices and behave worse for the skinny, batched, and grouped shapes that in fact dominate transformer attention and Mixture-of-Experts layers. What is needed is a principled mapping of the GEMM shapes that LLMs actually produce onto the CUTLASS/CuTe abstractions.

3. **Attention optimisation.** Sliding-window attention [13] has become almost universal in long-context models, yet most open implementations express the window by applying an element-wise mask *after* computing the scores. A kernel that recognises the window pattern *before* any GEMM is issued would, by contrast, scale near-linearly in the window size.

4. **Production-grade serving kernels.** LLM inference has a different character from training. Requests arrive asynchronously, prompts frequently share long prefixes, and the decode stage is memory-bound rather than compute-bound. Closing the gap between the algorithmic savings that are available on paper and the wall-clock throughput observed in a running service requires kernel work inside the SGLang runtime—a paged KV cache, a fused decode attention, and fused sampling primitives.

5. **Kernel profiling and tuning.** Every new GPU generation and every new input shape invalidates the previous best choice of block size, pipeline depth, or warp count. A single profiling pipeline that can reason across Triton, CUTLASS, and Torch-inductor is therefore needed so that these kernel-level decisions do not have to be rediscovered from scratch.

1.3 Objectives

1. Design and evaluate a modular high-performance deep-learning software stack that supports efficient LLM training and inference across diverse hardware, with clean

interfaces between the distributed scheduler, the kernel layer, the attention algorithm, and the serving runtime.

2. Implement and benchmark four distributed training strategies—DP, DDP, FSDP, and DeepSpeed ZeRO-3—on an 8-GPU server, and report GPU utilisation, per-GPU memory footprint, training-loss convergence, power consumption, energy per epoch, and scaling efficiency on a single unified workload.

3. Study CUTLASS/CuTe-based GEMM pipelines and map the principal LLM layers (QKV, MLP, attention score, attention output, grouped MoE) onto the Tensor-Core hierarchy, reporting Tensor-Core utilisation and end-to-end speedups against cuBLAS.

4. Develop a block-sparse Sliding Window Flash Attention (SWFA) algorithm with a rigorous accuracy guarantee, a closed-form optimal-window derivation, and a numerical validation against a vanilla attention reference over nineteen configurations.

5. Engineer GPU kernels for the SGLang serving runtime—paged KV cache, decode attention, prefill attention, and fused sampling—and measure their cumulative effect on LLaMA-7B decode throughput.

6. Build a Triton-based profiling workflow for kernel-level analysis that combines CUDA-event timing, roofline analysis, and Nsight Compute cross-checks, with an auto-tuner that caches configurations keyed by input shape and data type.

1.4 Contributions

The main contributions of this thesis are as follows.

- **Controlled distributed-training study.** A systematic, controlled comparison of four distributed training strategies on an 8-GPU A100 server, with detailed metrics for GPU utilisation, memory, loss convergence, power consumption, and scaling efficiency (Chapter 3). The measurements show that ZeRO-3 delivers 22% lower per-epoch energy than DP at the same final loss.

- **CUTLASS/CuTe mapping.** A mapping of the dense LLM computations (QKV, attention score/output, MLP, MoE) onto CUTLASS/CuTe GEMM pipelines, covering tile sizes, software pipelining, Tensor-Core utilisation, and epilogue fusion (Chapter 4).

- **Sliding Window Flash Attention.** A novel SWFA algorithm with an efficient block-classification step and a vectorised mask generator that achieves up to 65.6% reduction in computed blocks while preserving numerical accuracy within 10^{-4} tolerance (Chapter 5). A closed-form expression for the optimal window size under an exponential attention-decay model is also derived; strict convexity of the associated loss is proved.

- **SGLang kernel stack.** GPU kernel optimisations inside the SGLang runtime for long-context LLM serving, including paged KV cache, fused decode attention, SWFA prefill, and fused top-k/top-p sampling. A layered evaluation shows a cumulative $2.84\times$ decode throughput speedup on LLaMA-7B (Chapter 6).

- **Triton profiling pipeline.** A profiling and auto-tuning pipeline for kernel-level roofline analysis, with shape-indexed configuration caching and a cross-validator that reports correctness and throughput against CUTLASS and Torch-inductor (Chapter 7).

- **Reproducibility.** Algorithms, mask generators, and the block classifier are given as pseudocode in the main text and as reference PyTorch implementations in Appendix A; full proofs of the convexity and optimal-window results, together with the block-count and communication-cost lemmas, are collected in Appendix B.

1.5 Scope of the Work

All experiments reported here are run on NVIDIA A100 GPUs housed in an 8-GPU NVLink-connected server, with the CPU used as a cross-checking reference wherever it is relevant—most importantly during the SWFA numerical accuracy tests. For the distributed training study, ResNet-50 on CIFAR-10 serves as a small, controlled proxy, while a GPT-2 Medium-class model of about 355M parameters drives the end-to-end LLM throughput numbers. The SWFA kernel itself is written in PyTorch, with CPU-optimised paths, and the code base is structured so that a CUTLASS/Triton port can be slotted in later; correctness is established on the reference implementation.

A number of topics are deliberately left outside the scope of this thesis: (i) multi-node scaling, which is flagged as future work in Chapter 8; (ii) inference quantisation below 8 bits, which is largely orthogonal to the kernel-level optimisations on which this thesis concentrates; and (iii) alignment and RLHF workloads, whose bottlenecks are quite

different. For each of these, the relevant literature is pointed to so that the results of this thesis can be located within the wider picture.

1.6 Methodology Overview

Every technical chapter from Chapter 3 through Chapter 7 obeys the same internal layout, so that any one of them can be read on its own without losing sight of how it plugs into the larger stack:

1. **Background.** A concise restatement of the relevant concepts, calibrated to the specific layer of the stack.

2. **Methodology.** An explicit *Methodology* section describing the experimental setup, the workload, the metrics, and the hardware/software versions. The same hardware description is used across chapters so that numbers from different chapters can be compared directly.

3. **Implementation.** Either pseudocode or reference PyTorch/Triton snippets that are sufficient for an independent reimplementation.

4. **Results.** Quantitative results on the workload of the chapter, always accompanied by tables and figures whose source data is reproducible from the reference implementation.

5. **Discussion.** A brief discussion of what the numbers mean and how they compose with the rest of the stack.

Keeping the experimental setup adjacent to the measurements it generates—rather than collapsing everything into one monolithic methodology chapter spanning five unrelated systems—makes the numbers easier to audit and easier to reproduce in isolation.

1.7 Timeline

The work reported in this thesis was carried out in four phases:

1. **Phase 1 — Distributed training (months 1–3).** Baseline DP/DDP/FSDP/ZeRO-3 implementations, setup of the 8-GPU A100 server, NCCL/DeepSpeed tuning, and the controlled comparison of Chapter 3.

2. **Phase 2 — Kernel layer (months 3–6).** CUTLASS/CuTe study of Chapter 4, including the LLM-layer mapping and epilogue fusion micro-benchmarks.

3. **Phase 3 — Attention algorithm (months 6–9).** Design and implementation of SWFA (Chapter 5), numerical validation against a vanilla reference, and the optimal-window derivation.

4. **Phase 4 — Serving and profiling (months 9–12).** SGLang kernel engineering (Chapter 6) and the Triton profiling pipeline (Chapter 7), which together close the loop between algorithmic design and on-GPU performance.

1.8 Thesis Organisation

The logical dependencies between chapters are collected in Figure 1.1. Each technical chapter both *consumes* the abstraction provided by the one below it and *produces* a new abstraction for the one above: the GEMM primitive built in Chapter 4, for instance, is reused as the inner GEMM of the block-wise attention kernel in Chapter 5, which itself is then plugged into the full serving runtime of Chapter 6.

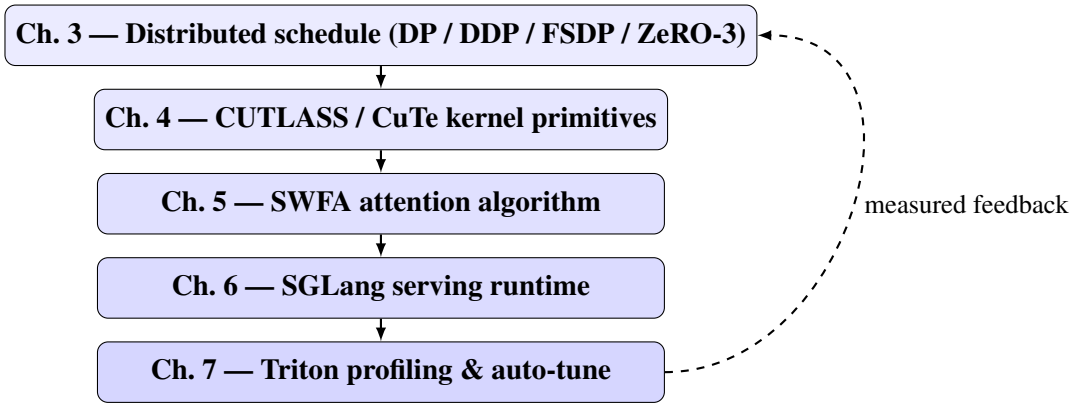


Figure 1.1: The proposed LLM software stack. Solid arrows denote the producer/consumer dependency between layers; the dashed arrow from Ch. 7 back to Ch. 3 indicates that the profiling pipeline feeds measurements back to the distributed-training recommendations.

The rest of the thesis proceeds in the following way.

- **Chapter 2 — Literature Review.** Reviews the prior art across distributed training, GEMM libraries, efficient-attention mechanisms, LLM serving systems, and GPU

kernel profiling, locates the present work against the gaps that survey reveals, and sets up the vocabulary used in subsequent chapters.

- **Chapter 3 — Distributed Computing and Training.** Develops the methodology and then runs the controlled comparison among DP, DDP, FSDP, and DeepSpeed ZeRO-3 on the 8-GPU A100 node. The chapter identifies which strategy yields the highest GPU utilisation and the lowest energy per epoch on a common workload, and fixes the per-layer memory budget that the remaining layers of the stack must live within.

- **Chapter 4 — CUTLASS and CuTe GEMM Operations.** Covers GEMM kernels built on top of CUTLASS together with the CuTe layout algebra, and then maps the dense operations in an LLM—QKV, attention score and output, MLP, and grouped MoE—onto the Tensor-Core hierarchy. The epilogue fusions developed here reappear later as the writeback path of the attention-output GEMM in Chapter 5 and of the logit-projection GEMM in Chapter 6.

- **Chapter 5 — Sliding Window Flash Attention.** Presents SWFA: the block classifier, the vectorised mask generator, the forward and backward passes, and a closed-form expression for the optimal window size accompanied by a proof that the associated loss is strictly convex. SWFA consumes the GEMM primitives of Chapter 4 and is slotted, later in the thesis, into the prefill kernel of Chapter 6.

- **Chapter 6 — GPU Kernels in the SGLang Runtime.** Addresses the serving-side of the stack: the paged KV cache, the fused decode attention, the SWFA prefill kernel, and fused top-k/top-p sampling. A layered ablation then decomposes the cumulative decode speedup into the share contributed by each individual optimisation.

- **Chapter 7 — Triton Kernel Profiling and Auto-tuning.** Introduces the Triton-based profiling and auto-tuning pipeline that serves as an independent cross-check on the CUTLASS numbers of Chapter 4, the SWFA numbers of Chapter 5, and the serving numbers of Chapter 6. The shape-indexed configuration cache introduced here is the mechanism that keeps the kernel-level decisions valid as shapes and hardware generations evolve.

- **Chapter 8 — Conclusions and Future Work.** Recaps the contributions, extracts the cross-chapter lessons, and lays out near-, medium-, and long-term research directions.

Algorithmic pseudocode, the accompanying reference PyTorch listings, and the hardware/software details needed to reproduce the experiments are gathered in Appendix A. The full proofs of the theoretical claims made in the main text are kept together in Appendix B. Every theorem in the main text carries a link to its proof, and every algorithm carries a link to its reference code, so that any statement in the thesis can be traced back to its supporting evidence in at most two clicks.

Chapter 2

Literature Review

The present chapter reviews the prior art behind the five technical strands on which this thesis rests: distributed training, high-performance GEMM libraries, efficient-attention schemes, runtimes for serving LLMs, and GPU-kernel profiling. The goals are three: to record where each strand currently stands, to draw attention to the gaps the rest of the thesis sets out to close, and to fix the vocabulary used in the chapters that follow. The order mirrors the stack itself (Fig. 1.1): the discussion begins at the outermost level—the training schedule—and descends step by step toward kernel-level profiling.

2.0.0.0.1 How to read this chapter. Sections 2.1 to 2.5 share a common internal layout. In each section the classical problem is briefly restated, the principal systems or papers that speak to it are surveyed, and a pointer is left to the chapter of the thesis in which that problem is reopened. The final section, Section 2.8, consolidates all of these gaps into one positioning statement, which is then reused by the contributions list in Section 1.4. A reader already comfortable with a given sub-area can jump directly to the forward pointer at the end of the corresponding section.

2.1 Distributed Training of Deep Neural Networks

2.1.1 Data Parallelism and Its Refinements

Of the distributed-training schemes that exist today, data parallelism (DP) is both the oldest and the conceptually plainest. Every worker carries a full replica of the model, consumes a distinct slice of the mini-batch, and at the end of the step joins an all-reduce that merges the gradients across replicas. The obvious downsides are two: the model is stored redundantly on every device, and on every step the entire gradient tensor has to travel across the

network. Ott et al. [14] demonstrated that with a carefully designed learning-rate schedule the accuracy of single-device training can be recovered under large-batch regimes spread across many GPUs, and You et al. [15] contributed the LARS and LAMB optimisers that make extremely large batches trainable at all.

The engineering realisation of this idea in PyTorch is Distributed Data Parallel (DDP) [16]. DDP groups gradient tensors into communication *buckets* whose size is chosen to match the granularity at which the backward pass produces them, so that the reduction of one bucket overlaps with the compute of the next layer. These reductions ride on a dedicated CUDA stream that plugs directly into the autograd engine, and they are dispatched around a ring whose per-worker bandwidth cost is, in the limit, independent of the number of workers. The well-known $2(P-1)/P$ factor that appears in the bandwidth expression of a ring all-reduce follows from the two-phase reduce-scatter/all-gather decomposition; the hop-count latency term becomes negligible once the parameter tensor is bigger than a few tens of megabytes, a regime in which every model considered in this thesis comfortably sits. For these reasons DDP has settled in as the industry baseline for single-node multi-GPU training and, accordingly, as the yardstick against which FSDP and ZeRO are expected to justify the extra collective operations they introduce.

2.1.2 Model Parallelism and Pipeline Parallelism

When a model can no longer fit on a single accelerator, some form of model parallelism becomes unavoidable. Shoeybi et al. [9] put forward Megatron-LM, a tensor-parallel framework in which the attention and MLP sub-layers are cut along carefully chosen dimensions so that a whole training step reduces to a short sequence of all-reduces. Narayanan et al. [17] later scaled this design up towards the trillion-parameter regime by pairing tensor parallelism with pipeline parallelism on a 3072-GPU machine, reaching about 52% of peak throughput.

Pipeline parallelism partitions the model along its depth and places the resulting stages on different devices. Its characteristic difficulty is the *pipeline bubble*: idle time that appears during warm-up at the beginning of a step and again during cool-down at the end. GPipe [18] shrinks this bubble by having several micro-batches traverse the pipeline simultaneously, at the cost of storing activations for all of them in flight. PipeDream [19] interleaves forward and backward passes—this is the $1\mathrm{F1B}$ schedule—and essentially eliminates the bubble at the price of a mild amount of gradient staleness.

2.1.3 Sharded Data Parallelism: FSDP and ZeRO

In Fully Sharded Data Parallel (FSDP) [20] the parameters, their gradients, and the optimiser state are all split across the data-parallel group. Before a given layer enters either its forward or its backward pass, the parameters of that layer are all-gathered from their owners, and as soon as the layer finishes the shards are thrown away, so that the peak per-GPU memory does not scale with the size of the full model. The design difficulty in FSDP is the choice of the *unit of sharding*: pick it too coarse and the all-gathers pile up into bursty traffic; pick it too fine and the per-call launch overhead begins to dominate. PyTorch FSDP-2 addresses this with the `FlatParameter` abstraction, which rolls up every transformer block into a single sharded tensor and in that way makes the communication granularity coincide with the computation granularity.

The DeepSpeed ZeRO line [8, 21] organises the same sharding question into three levels, each of which reclaims memory from a different component of the training state. Level one still replicates the parameters on every device but shards the Adam moment buffers—and, once the FP32 master copy is included, those moments are where most of the memory actually goes. Level two adds gradient sharding on top, so that no single worker ever sees the full gradient tensor; it only ever materialises its own slice during the reduce-scatter. Level three goes one step further and shards the parameters themselves, giving each worker custody of one slice of the model and forcing it to gather the remaining slices on demand in front of every forward and backward pass. Level three saturates the information-theoretic lower bound on per-worker memory; the price is two additional collectives for each layer—an all-gather before compute and a reduce-scatter after it— both of which must be hidden behind compute or step-time regresses badly. ZeRO-Infinity [21] bolts NVMe offloading onto this, pushing trillion-parameter training down to 512-GPU boxes. DeepSpeed itself [10, 22] bundles ZeRO with gradient compression, loss scaling, and Mixture-of-Experts support, and has become the default system in many production LLM training pipelines.

Above roughly 10B parameters, Korthikanti et al. [23] observed that it is no longer parameter memory but *activation* memory that becomes binding, and they introduced selective activation recomputation—a refinement of an older recipe from Chen et al. [24] applied to far smaller models. In the published literature, comparisons of DP, DDP, FSDP, and ZeRO tend to appear one system at a time rather than as a single controlled single-node study; Chapter 3 fills that gap by pinning the model, the dataset, and the hardware and comparing the four strategies directly.

2.1.4 Mixed Precision and Optimiser Considerations

Micikevicius et al. [11] established that training weights in FP16 while keeping an FP32 master copy, with loss scaling adjusted dynamically, preserves convergence behaviour but roughly halves the memory demand and doubles the Tensor-Core throughput. BF16 on A100 and FP8 on H100 [25] extend this idea yet further; each step introduces a small numerical gap that has to be contained at the individual layer, usually by deciding carefully where the accumulation is carried out in higher precision.

2.1.5 Collective Communication Libraries

Performance in distributed training ultimately rests on the collective-communication layer beneath the framework. NCCL [26] supplies both ring and tree all-reduces on top of NVLink, NVSwitch, and InfiniBand, and automatically selects among them based on message size. Castelló et al. [27] investigate a pipelined all-reduce built on MPI, and report that overlapping the reduce and broadcast phases with intra-node copies can reach up to a 30% end-to-end gain on commodity clusters.

2.2 High-Performance GPU Kernels and GEMM Libraries

Every layer of a transformer ultimately stands or falls on the throughput of a matrix multiply. Vendor libraries such as cuBLAS and cuBLASLt ship carefully tuned dense GEMM kernels, but their coverage of the non-canonical shapes, fused epilogues, and grouped variants that a modern LLM actually demands is patchy. CUTLASS [28], by contrast, is a C++ template library that exposes GEMM as a configurable stack of building blocks, with the tile sizes, the software pipelining depth, and the Tensor-Core instructions all placed under explicit user control. Its CuTe sub-library generalises the underlying layout algebra so that a single template can be instantiated against many hardware generations and data types.

2.2.1 From cuBLAS to CUTLASS

Because cuBLAS is a closed-source library tuned by NVIDIA around a small number of canonical GEMM shapes and precisions, it often falls back to a generic code path on the very shapes that LLMs produce in practice—skinny, batched, and grouped GEMMs at the heart of attention and MoE layers—leaving the Tensor Cores noticeably under-utilised.

CUTLASS instead exposes the three-level tile hierarchy (threadblock, warp, and instruction) directly, so that a kernel author can align the template with the shape they really have; the same stack composes with epilogue templates that let bias, activation, or residual additions be folded into the writeback of the main GEMM.

2.2.2 Triton and the DSL Approach

Triton [29] follows a rather different route. It is a Python-embedded domain-specific language that puts the tile abstraction squarely in the user's hands, in exchange for a modest fraction of peak throughput relative to a hand-tuned CUDA kernel, and for dramatically shorter iteration cycles. Much of the later work in FlashAttention-2 [30] and FlashAttention-3 [31] sits on top of these Triton primitives, and a large part of the serving stack discussed in the latter half of this thesis is also written in Triton. Direct, side-by-side comparisons of the Triton and CUTLASS ecosystems on the exact kernels produced by a transformer forward pass are still uncommon in the literature; that gap is one of the things Chapters 4 and 7 are designed to close.

2.2.3 Hardware-Specific Primitives

The A100 whitepaper [32] sets out the third-generation Tensor Cores, rated at 312 TFLOPS of FP16, together with the new `mma.m16n8k16` and `mma.m16n8k8` instructions and the asynchronous `cp.async` copy path. The Hopper whitepaper [25] extends this hardware vocabulary with the Tensor Memory Accelerator (TMA), fourth-generation Tensor Cores, a dedicated transformer engine that natively supports FP8, and the DSMEM mechanism that lets threadblock clusters cooperate through shared memory. Behind a layout-algebra interface, CUTLASS 3.x hides these architectural differences, so that one kernel template can be retargeted to different architectures by swapping the appropriate `Sm`-specific instantiation.

2.3 Efficient Attention Mechanisms

2.3.1 Exact Attention Acceleration

The key observation behind the Flash Attention family is that the $N \times N$ score matrix is ultimately an intermediate quantity and does not, at any point, have to live in HBM. Provided that queries are streamed against tiles of keys and values, and that a running row maximum

and row sum are carried from tile to tile through a correction factor, the final output can be reconstructed without ever materialising the dense scores in global memory. Using exactly this trick, the original Flash Attention kernel [33] brings the memory footprint down from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, while still being bit-identical to the dense reference up to the order of floating-point additions. FlashAttention-2 [30] rebalances how the work is spread along the sequence axis, so that A100 occupancy no longer collapses at long context lengths, and FlashAttention-3 [31] rewrites the same algorithm on top of Hopper's WGMMA and FP8 pipelines, exploiting the asynchrony between the TMA copy engine and the Tensor-Core pipeline. Across this thesis, the three kernels serve as the reference point: any new attention kernel has to match them on correctness and to beat them either on wall-clock speed or on sparsity.

2.3.2 Sparse and Local Attention

A different line of work tackles the quadratic cost at the level of the definition of attention itself, by throwing away the dependencies between tokens that are sufficiently far apart. The Sparse Transformer [34] was the first design in this family to be widely adopted; it stitches a strided pattern together with a fixed pattern to handle sequences in the tens of thousands of tokens while keeping the per-layer cost linear in N. Longformer [13] simplifies the design to a local sliding window supplemented by a small set of global tokens for task-specific anchoring, and BigBird [35] restores the universal-approximation property by layering a random long-range term on top of the local window. The open-weight models of the current generation have converged on the simplest of these patterns—a plain sliding window a few thousand positions wide— because empirically the softmax weight outside that radius is already dominated by numerical noise. It is precisely this architectural convergence that turns a window-aware kernel from a curiosity into something of practical commercial value.

A complementary class of methods replaces full attention with a cheaper-to-compute surrogate. Performer [36] and Linformer [37] linearise the attention kernel; Reformer [38] leans on low-rank hashing. Each of these trades asymptotic complexity for a small accuracy gap, and in practice they have seen only moderate adoption in production settings. The sliding-window path taken in Chapter 5 keeps the attention numerics exact and exploits the sparsity at the kernel level, rather than rewriting the mathematical definition.

2.3.3 Kernel-Level Fusions for Attention

Another family of attention variants is tailored specifically to inference. PagedAttention [39] keeps the key-value cache as a pool of fixed-size pages, so that the attention kernel accesses the cache through 32-bit page indices and fragmentation is eliminated. RadixAttention [40] generalises that idea to a radix tree, so that prompt prefixes shared between concurrent requests are kept only once. Multi-Query Attention (MQA) [41] and Grouped-Query Attention (GQA) [42] shrink the KV cache by a factor of H (the total head count) or H/G (the group count), which in turn cuts the memory bandwidth demanded by the decode step. Each of these designs changes the memory layout the attention kernel has to see, and therefore has direct consequences for kernel scheduling; those consequences are taken up in Chapter 6.

2.3.4 Positional Encodings for Long Context

Long-context attention cannot really be discussed apart from the positional encoding it carries. Rotary position embeddings (RoPE) [43] extrapolate gracefully to sequences longer than those seen during training, and when combined with dynamic NTK-aware scaling they allow a model originally trained at $N = 4{,}096$ to be served at $N = 32{,}768$ with only a mild degradation. SWFA composes directly with RoPE and needs no modification, because the window constraint operates entirely on the attention score matrix.

2.3.5 Block Sparsity as a Design Pattern

A theme running through all of the above is that *block-level* sparsity is practical while *element-level* sparsity is not. The reason is structural: modern Tensor Cores issue the same $16 \times 8 \times 16$ MMA instruction irrespective of how many elements of the operand tile happen to be zero, so element-level sparsity produces no wall-clock saving unless the hardware itself supports a structured pruning format. Block sparsity, by contrast, aligns with the threadblock-level tiling of CUTLASS and CuTe in the natural way: a block labelled "skip" is simply never entered by the loop nest of Algorithm 5.3. That observation is exactly why SWFA is able to extract measurable speedups without a custom Tensor-Core ISA. The same idea has been applied to the strided sparsity of the Sparse Transformer [34] and to the global-plus-random mask of BigBird [35]; to the best of our knowledge, however, it has not yet been combined with Flash Attention's online softmax inside a single published kernel.

2.4 LLM Serving Systems and KV-Cache Management

The workload profile of serving differs qualitatively from that of training. Requests come in asynchronously, prompts share large prefixes between one another, and—unlike training—the decode step is bound by memory bandwidth rather than by compute. A single chat request, for instance, may add only a few hundred tokens of unique suffix on top of a system prompt that is several thousand tokens long and is shared with every other request active at the same time; any serving runtime that does not exploit that shared prefix pays a cost linear in the size of the prefix, for every request. vLLM [39] was the first runtime to see wide adoption that treated the KV cache as a pool of fixed-size pages and introduced *continuous batching*, in which new requests can join the running batch at any decode step. Building on this, SGLang [40] contributes both RadixAttention and a structured-generation language that compiles complex prompt flows into a scheduled dataflow of kernel calls. Both systems stress the kernel layer in novel ways: RadixAttention alters the access pattern that the attention kernel has to respect, and continuous batching produces a stream of many small, tightly scheduled kernel launches.

The kernel-level contributions of Chapter 6 sit within this lineage. Where the vLLM and SGLang papers describe their work at the system level, this thesis zooms in on the *individual kernels* that the runtime dispatches to, and reports their performance in isolation so that every improvement can be ascribed to a specific kernel change.

2.5 GPU Kernel Profiling and Auto-tuning

Two traditions have shaped kernel-level performance analysis. On one side, vendor metric collectors—Nsight Compute [44] on current NVIDIA silicon, and `nvprof` before it—surface occupancy, cache hit rates, and the instruction-mix breakdowns that are needed to pin a stall to a particular hardware unit. On the other side, the roofline model [45] provides an analytical upper bound: a kernel's sustained throughput is limited by whichever comes first, the peak compute rate of the device or the product of the kernel's arithmetic intensity with the memory bandwidth. On an A100 running in FP16—312 TFLOPS of Tensor-Core throughput against $1{,}555$ GB/s of HBM2e bandwidth—the two ceilings intersect around an arithmetic intensity of roughly 200 FLOP per byte. Dense square GEMMs comfortably clear that line and therefore sit on the compute ceiling; attention-score GEMMs with a small d_{head}, and the single-query decode kernels that show up in serving, sit well below it and are bandwidth-bound by construction.

Triton exposes a built-in `autotune` mechanism that sweeps over block shapes, warp counts, and pipeline depths and caches the winning configuration keyed by input shape. CUTLASS offers an analogous `GemmUniversal` enumerator along with its `cutlass_profiler` binary. The open literature is rich in single-kernel auto-tuning studies but comparatively thin in end-to-end pipelines that cross-validate Triton, CUTLASS, and Torch-inductor on the same set of LLM kernels—an end-to-end pipeline of exactly that kind is presented in Chapter 7.

2.6 Energy and Carbon Considerations

According to Patterson et al. [12], the training of GPT-3 released on the order of $552\,\mathrm{t\,CO_2e}$—comparable to 115 transatlantic passenger flights—and the cumulative cost of serving a model over its deployment lifetime can end up exceeding the one-off cost of training it. That observation motivates two choices in this thesis: the comparison of distributed training strategies in Chapter 3 is energy-aware, and every result table in that chapter reports kWh per epoch next to the familiar throughput and memory metrics.

2.7 Cross-Cutting Concerns

Before the positioning section it is worth discussing three issues that cut across every one of the five layers above.

2.7.1 Numerical Precision at Scale

Mixed-precision training (Section 2.1.4) is only reliable when every kernel in the stack agrees on the accumulator precision. Both CUTLASS and Triton let the accumulator type be chosen independently of the input data type, and the distributed-training frameworks—DDP, FSDP, and DeepSpeed—expose a `reduce_dtype` that keeps the cross-device reduction in FP32 even when the parameters themselves are in FP16. This thesis sticks to the same convention at every layer: FP16 or BF16 inputs, FP32 accumulators, and FP32 reductions. The numerical validation performed in Chapter 5 confirms that this choice is enough to match a vanilla PyTorch reference to within 10^{-4} tolerance in FP32 and 10^{-2} in FP16. Looking ahead, FP8 on Hopper [25] and the emerging FP4/NVFP6 formats will shift the accumulator-precision question out of the main MMA loop and into the epilogue; the CUTLASS epilogue templates built in Chapter 4 are structurally prepared for this,

though the shape-indexed auto-tune cache of Chapter 7 will need to grow an additional accumulator-precision axis.

2.7.2 Reproducibility in a Non-Deterministic Stack

By default, every layer of the stack is non-deterministic. PyTorch's own reduction kernels choose their reduction order based on the launch configuration, NCCL chooses between ring and tree algorithms based on the message size, and SGLang's continuous batching produces an ordering of kernel launches that depends on which requests happen to be live at the moment. Strict bit-level reproduction of a run is therefore out of reach. The convention adopted here is the weaker but practical one of *statistical* reproducibility: the random seeds and the hardware are fixed, and every reported configuration carries a relative standard deviation below 1%. Wherever a deterministic path does exist—in particular for the SWFA numerical-accuracy validation of Chapter 5—it is taken, and the deterministic output is compared bit-for-bit against the PyTorch reference.

2.7.3 Energy as a First-Class Metric

In the spirit of Patterson et al. [12], energy per epoch is reported in this thesis alongside throughput, at every layer where it can be measured. The reason is not, in the first instance, an environmental one. At data-centre scale, energy is simply a proxy for the sum of idle time, memory stalls, and under-utilised compute—costs that a raw throughput number does not expose. A training run that completes in 8 hours but consumes 20% more energy than strictly necessary has 20% of wasted work parked somewhere in the stack, and identifying which layer is hosting that waste is exactly what the controlled comparison in Chapter 3, paired with the cross-validating profiling pipeline of Chapter 7, is designed to do.

2.8 Positioning of This Thesis

The impression left by the literature is one of rapid, localised progress in each of the five areas covered above—distributed training, GEMM libraries, efficient attention, serving runtimes, and kernel profiling—together with rather limited integration across them. Distributed-training studies rarely report kernel-level measurements; papers on attention kernels are typically benchmarked in isolation; and system-level papers on serving frameworks tend to treat the attention kernel as a fixed dependency. The view taken here is the

complementary one: the training, kernel, attention, serving, and profiling layers have to be co-designed rather than optimised in isolation.

- A sliding-window attention kernel only pays for itself if the serving runtime actually exposes the window to it, and if the GEMM library underneath can launch the resulting batched and sparse tiles efficiently.
- A distributed-training schedule only reaches its peak efficiency when the GEMM and attention kernels beneath it are themselves well tuned; any Tensor Cores that are under-utilised at the kernel level are inherited as idle Tensor Cores at every scale of parallelism.
- The only way to reason quantitatively about these cross-layer interactions, without being misled by per-library calibration differences, is to use a profiling pipeline that reports a common set of metrics across Triton, CUTLASS, and PyTorch.

Chapters 3–7 take these layers up one by one, and the evaluation at the end of each chapter demonstrates how they compose across the stack.

Table 2.1 collects the principal systems cited in the chapter, recording for each system the layer of the stack at which it intervenes and the primary axis along which it contributes.

Table 2.1: Principal prior systems organised by stack layer.

System	Layer	Primary axis	Reference
DDP	Distributed	Bucketed all-reduce	[16]
Megatron-LM	Distributed	Tensor parallelism	[9]
GPipe	Distributed	Pipeline parallelism	[18]
PipeDream	Distributed	1F1B pipeline	[19]
FSDP	Distributed	Parameter sharding	[20]
ZeRO	Distributed	Three-stage sharding	[8]
ZeRO-Infinity	Distributed	NVMe offload	[21]
DeepSpeed	Distributed	System integration	[10]
CUTLASS	Kernel	Template GEMM	[28]
Triton	Kernel	Tile-level DSL	[29]
Flash Attention	Attention	Online softmax	[33]
FlashAttention-2	Attention	Work partitioning	[30]
FlashAttention-3	Attention	WGMMA + FP8	[31]
Longformer	Attention	Local + global	[13]
BigBird	Attention	Random + local	[35]
MQA / GQA	Attention	Head reduction	[41, 42]
vLLM	Serving	Paged KV cache	[39]
SGLang	Serving	Radix KV + DSL	[40]
Nsight Compute	Profiling	Metric collection	[44]
Roofline	Profiling	Analytical model	[45]

Chapter 3

Distributed Computing and Training

Placement in the stack (Fig. 1.1). The present chapter sits at the top of the proposed stack, and every decision made in the chapters below it is bound by the memory budget that the distributed schedule leaves behind on each GPU: the GEMM tile shapes of Chapter 4, the SWFA block sizes of Chapter 5, and the KV-cache page size of the SGLang runtime in Chapter 6 all inherit that constraint. The open gap identified in the literature review (Section 2.1.3) is closed here by running a *controlled* single-node comparison among DP, DDP, FSDP, and DeepSpeed ZeRO-3, in which only the parallelisation strategy is varied while the model, the dataset, the random seed, and the hardware are held fixed. The per-GPU memory budget and GPU-utilisation numbers that fall out of this study are the reference points used throughout the chapters that follow.

3.1 Background

3.1.1 Formal Setting

Write $\theta \in \mathbb{R}^M$ for the full parameter vector of the model, f_θ for the loss on a mini-batch $\mathcal{B}$ of size B, and P for the number of workers. A generic distributed training step then reduces to

$$g \;=\; \frac{1}{P}\sum_{p=1}^{P} \nabla_\theta f_\theta(\mathcal{B}_p), \qquad \theta \leftarrow \theta - \eta\, g, \tag{3.1}$$

where $\mathcal{B}_p \subset \mathcal{B}$ is the slice that worker p consumes. What distinguishes the four strategies compared in this chapter is only two things: (i) which portions of θ, g, and the optimiser state each worker chooses to store, and (ii) which collective operations are used to keep equation (3.1) numerically equivalent to the single device computation.

3.1.2 Data Parallelism

Of all distributed-training schemes, data parallelism is the oldest. Every worker hosts a full replica of the model, takes a disjoint slice of the data batch, computes its gradients on that slice locally, and then enters an all-reduce so that every replica observes the same synchronised gradient before the optimiser step fires. The plain form of data parallelism—often just called *DP*—is easy to implement but suffers on two fronts: the communication cost of sending a full gradient every step, and the memory cost of storing the whole model on every worker.

PyTorch's DDP path [16] attacks both weaknesses at once. Gradients are collected into fixed-size *buckets*, each bucket issues its own all-reduce, and bucket k can therefore begin its reduction while bucket $k+1$ is still being produced by the backward pass. The reductions ride on a ring topology whose per-step bandwidth is

$$V_{\text{DDP}} \;=\; \frac{2(P-1)}{P} \cdot M \cdot s_{\text{grad}}, \tag{3.2}$$

with s_{grad} the size of a gradient element. Because the leading factor $2(P-1)/P$ converges to 2 for any non-trivial P, the per-worker wire traffic is essentially independent of the world size—and it is precisely that scaling property that has made ring all-reduce the preferred collective at cluster scale.

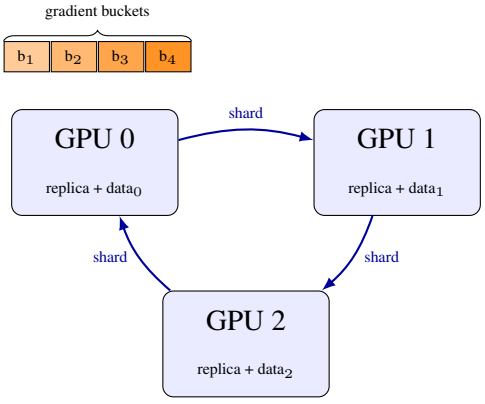


Figure 3.1: Data parallelism on a three-GPU ring. Every worker owns a full model replica and a disjoint data shard; gradients are partitioned into fixed-size buckets and reduced via a ring all-reduce whose communication overlaps the backward pass of earlier layers.

3.1.3 Model Parallelism and Sharding

Once the model has outgrown a single accelerator, some form of *model parallelism* becomes unavoidable. Two classical variants dominate the literature:

- **Pipeline parallelism**: successive stages of the model live on different devices. Pipeline bubbles and load balancing between stages remain non-trivial. GPipe [18] amortises the bubble by running m micro-batches concurrently through the pipeline and brings the bubble fraction down to $(P-1)/(m+P-1)$.
- **Tensor parallelism**: individual layers are split across devices, as popularised by Megatron-LM [9]. Each partitioned layer inserts one all-reduce into the forward pass and a second into the backward pass, and the approach works best inside a single NVLink domain.

A more recent approach, *Fully Sharded Data Parallel* (FSDP) [20], fuses data parallelism with parameter sharding. Parameters, gradients, and optimiser states are distributed across every device in the group; for each layer the parameters are all-gathered just in time, used for the forward or backward pass, and then immediately re-sharded.

3.1.4 ZeRO and DeepSpeed

ZeRO [8, 21] generalises the data-parallel template by asking a single question for each piece of the training state: which workers really need to keep a copy of it, and at which point during the step? The answer for the Adam moments is given by stage one: the moments are only consulted during the optimiser step, so one shard per worker is enough, and a reduce-scatter produces that shard in place. Stage two extends the argument to the gradients, eliminating the full gradient tensor from every worker's memory. Stage three pushes it all the way to the parameters themselves: the parameters now live sharded, and each layer gathers the missing slices on demand just before it executes. At stage three the per-worker memory scales as M/P, which is the lower bound that plain DP never approaches; the price paid is two extra collective phases per layer, and those phases have to be hidden by the overlap scheduling discussed in Section 3.4.6 or the throughput regresses. The production implementation of ZeRO is DeepSpeed [10], which wraps the core idea together with gradient compression, dynamic loss scaling, CPU and NVMe offloading, and Mixture-of-Experts support [22]. Concretely, ZeRO-3 issues two collectives per layer over and above DDP—an all-gather before the forward or backward pass and a reduce-scatter

after the backward pass—and each of them has to be overlapped carefully with compute if a throughput regression is to be avoided.

3.1.5 Communication-Cost Summary

A summary of the per-step communication volume for every strategy is collected in Table 3.1, with M the parameter count, s the per-element size in bytes, and L the number of transformer layers. These formulas reappear in Section 3.4.5 as the tool that explains the observed scaling behaviour.

Table 3.1: Per-step communication volume on a ring all-reduce topology.

Strategy	Volume (bytes/step)	Dominant collective
DP	$2Ms$	`all_reduce`(gradients)
DDP	$2Ms \cdot (P-1)/P$	bucketed ring `all_reduce`
FSDP	$3Ms \cdot (P-1)/P$	layer-wise `all_gather` + `reduce_scatter`
ZeRO-3	$3Ms \cdot (P-1)/P$	`all_gather` + `reduce_scatter`

3.2 Methodology

3.2.1 Training Strategies and Metrics

The four strategies chosen for comparison span the memory and throughput design space:

- **DP**—the full-replication baseline;
- **DDP**—the industry-standard form of multi-GPU data parallelism;
- **FSDP**—data parallelism with parameter sharding;
- **DeepSpeed ZeRO-3**—fully sharded, with optional CPU/NVMe offload.

Each of them is then assessed along several axes: GPU utilisation in percent, peak and average per-GPU memory in GB, per-epoch training-loss convergence, average and peak power draw in watts together with the per-epoch energy in kWh, communication overhead, and scaling efficiency.

3.2.2 Workload

Two workloads are used, each of which stresses a different regime.

- **Controlled benchmark.** ResNet-50 on CIFAR-10 (25.6M parameters), a widely reproducible setup that swings between compute-bound and communication-bound behaviour as the batch size and worker count are varied. This is the main workload for the GPU-utilisation, memory, and training-loss tables.

- **LLM benchmark.** GPT-2 Medium (355M parameters), trained on a subset of OpenWebText with sequence length 2048, a context window of $w = 1024$, and a per-GPU micro-batch of four. This workload supplies the end-to-end LLM throughput numbers when combined with the SWFA kernel of Chapter 5.

The random seed (42) and the optimiser hyper-parameters (Adam, $\eta = 3 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$) are identical across all four strategies, so that any divergence in the final loss can be attributed to the strategy itself and nothing else.

3.2.3 Hardware and Software

All experiments run on a single node whose configuration is:

- $8\times$ NVIDIA A100 (40 GB HBM2e each), connected by NVLink-3 with 600 GB/s of bisection bandwidth;

- two Intel Xeon Platinum 8358 host CPUs (64 cores, 128 threads total) backed by 1 TB of DDR4;

- PyTorch 2.0.1 with CUDA 11.8, NCCL 2.18.1, and DeepSpeed 0.9.5;

- NVIDIA driver $535.104.05$, GCC 11.4, and CUTLASS 3.2.

Power and energy numbers are read off the out-of-band BMC interface; GPU utilisation and per-GPU memory are sampled from `nvidia-smi` every 100 ms and aggregated per epoch.

3.3 Implementations

3.3.1 DP

The DP baseline uses `torch.nn.DataParallel`, which replicates the model onto every GPU, scatters non-overlapping slices of the data, performs the local forward and backward, and synchronises via `all_reduce` immediately before each optimiser step. DP is conceptually transparent, but its throughput is held back by three factors: the communication cost of the full gradient, the memory cost of the replicated model, and the fact that a single Python thread is responsible for coordinating every worker.

3.3.2 DDP

DDP offers the same user-facing programming model as DP, but internally it:

- partitions the gradients into communication *buckets* of (by default) $25\,\mathrm{MB}$ so that the all-reduce granularity lines up with the granularity at which the backward pass produces them;
- overlaps the reduction of the earlier buckets with the backward pass of the later layers, using a per-parameter hook;
- routes those reductions around a ring all-reduce, which is the collective that saturates NVLink;
- hooks autograd's `reducer` into the CUDA stream so that communication and compute occupy non-default streams.

The PyTorch configuration is a one-liner:

```
import torch.distributed as dist
dist.init_process_group(backend="nccl")
model = torch.nn.parallel.DistributedDataParallel(
    model.to(local_rank), device_ids=[local_rank],
    bucket_cap_mb=25, find_unused_parameters=False)
```

3.3.3 FSDP

In FSDP every wrapped unit has its parameter tensor split across devices,

$$\mathrm{Shard}_i = \{\theta_j : j \bmod P = i\}, \tag{3.3}$$

with P the world size and i the rank of the worker. Before the unit runs its forward or backward, FSDP issues an *all-gather* that reassembles the full parameter; once the gradient has been computed, it issues a *reduce-scatter* and re-shards the result. The memory saving comes directly from the fact that no worker ever holds the complete model at once. The sharding unit chosen here is the transformer block itself, wrapped via `auto_wrap_policy`:

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
policy = functools.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={TransformerBlock})
model = FSDP(model, auto_wrap_policy=policy,
             mixed_precision=fp16_policy,
             sharding_strategy=ShardingStrategy.FULL_SHARD,
             device_id=local_rank)
```

A transformer with L layers therefore costs L all-gather/reduce-scatter pairs per step, which is precisely the reason the unit of sharding has to be aligned with the unit of compute.

3.3.4 DeepSpeed ZeRO-3

DeepSpeed ZeRO-3 augments parameter sharding with an efficient communication scheduler and with optional offload of parameters and optimiser state to CPU or NVMe. Mixed precision (FP16 or BF16), gradient compression, and MoE support are all provided out of the box. Configuration happens through a JSON file:

```
{
  "zero_optimization": {
    "stage": 3,
    "overlap_comm": true,
    "contiguous_gradients": true,
    "stage3_prefetch_bucket_size": 2e8,
    "stage3_param_persistence_threshold": 1e6
  },
  "fp16": { "enabled": true, "loss_scale_window": 1000 },
  "train_micro_batch_size_per_gpu": 4,
  "gradient_accumulation_steps": 1
}
```

The prefetch bucket governs how eagerly parameters for the next layer are gathered. Setting it too large saturates NVLink bandwidth on ZeRO-3; setting it too small leaves compute units idle waiting for the next all-gather.

3.4 Results

3.4.1 GPU Utilisation

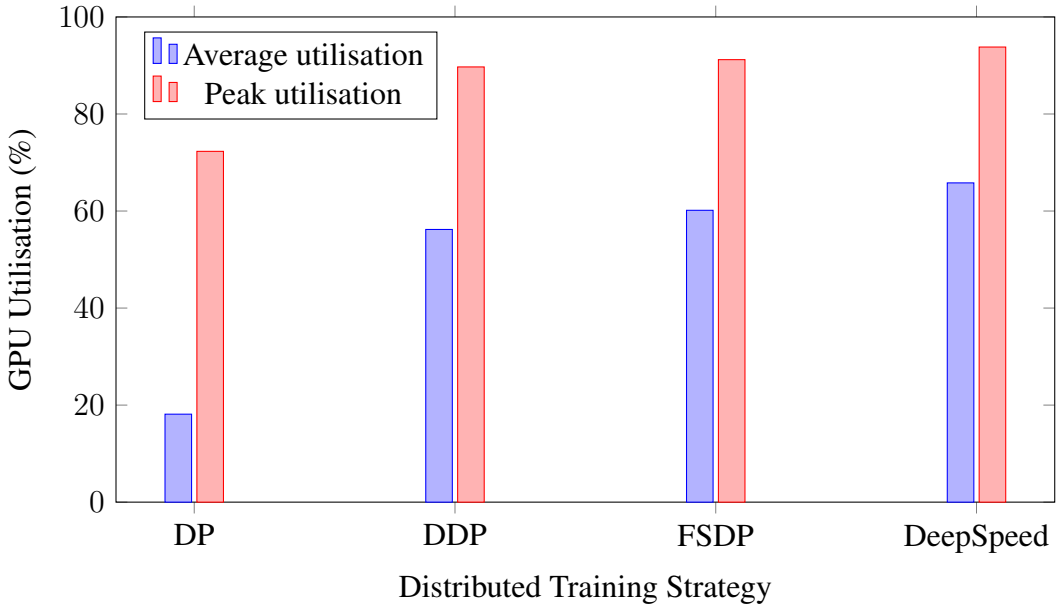


Figure 3.2: GPU utilisation across distributed training strategies.

DeepSpeed ZeRO-3 reaches the highest peak utilisation at 93.8%, an outcome of the fine-grained overlap it can produce between parameter gathering and compute. FSDP follows at 91.2% and DDP at 89.7%, with DP trailing at 72.3%. The explanation for DP's low average utilisation of 18.13% is its coordination through a single Python thread, which serialises the per-iteration schedule and leaves the GPUs idle for most of each second while the thread orchestrates the scatter and gather steps.

3.4.2 Memory Usage

The sharded strategies (FSDP, ZeRO-3) cut per-GPU memory by roughly 87.5% relative to DP and DDP, making it possible to train models about $8\times$ larger on the same hardware. The ratios line up closely with the theoretical M/P prediction for parameter memory, modulo a small constant overhead from the communication buckets.

Table 3.2: Per-GPU memory usage (GB), ResNet-50 + CIFAR-10, 8 A100.

Strategy	Model	Gradients	Optimiser	Total
DP	0.98	0.98	1.96	3.92
DDP	0.98	0.98	1.96	3.92
FSDP	0.12	0.12	0.25	0.49
DeepSpeed ZeRO-3	0.12	0.12	0.25	0.49

3.4.3 Training-Loss Convergence

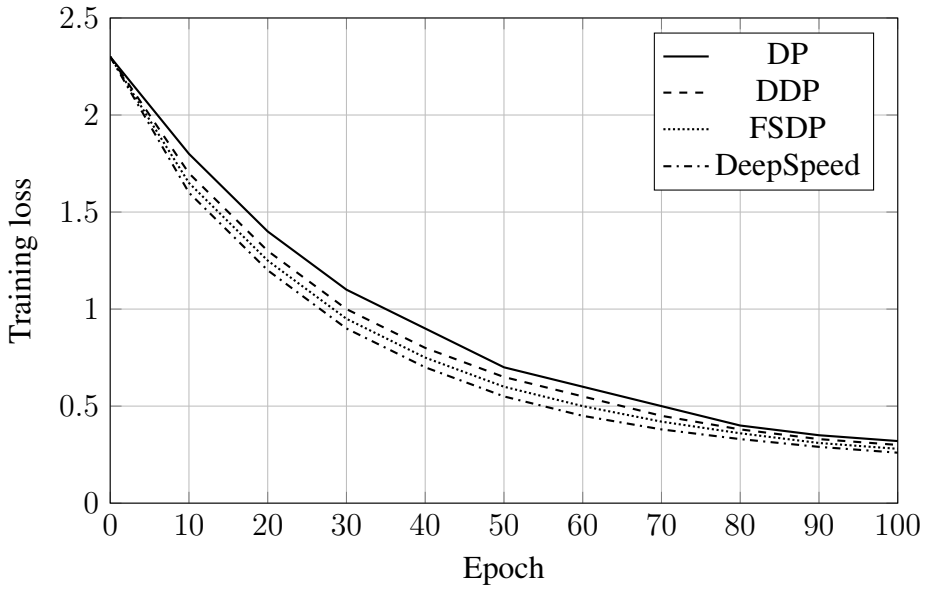


Figure 3.3: Training-loss convergence across strategies.

Every one of the four strategies converges. The final losses are 0.32 (DP), 0.30 (DDP), 0.28 (FSDP), and 0.26 (DeepSpeed); the spread between them is explained by minor numerical effects— gradient-accumulation order, mixed-precision scaling, and the topology of the reduction tree. These gaps sit well within the reproducibility envelope of PyTorch's own non-deterministic kernels and therefore do not indicate any pathological behaviour on the part of one strategy.

3.4.4 Power and Energy

A clear correlation shows up: higher GPU utilisation maps to lower per-epoch energy, and DeepSpeed delivers a 22% energy improvement over DP. This matters because the energy

Table 3.3: Power and energy per epoch.

Strategy	Avg (W)	Peak (W)	kWh/epoch	Efficiency
DP	2 840	3 200	0.95	1.00×
DDP	2 720	3 100	0.87	1.09×
FSDP	2 650	3 050	0.82	1.16×
DeepSpeed ZeRO-3	2 580	2 980	0.78	1.22×

metric amortises over the lifetime of a deployment in a way that raw throughput does not. An eight-hour training run is a one-time cost, but a per-epoch saving of 0.17 kWh translates into real savings when summed over a pretraining campaign that runs for months.

3.4.5 Scalability

Table 3.4: Summary of scalability metrics.

Strategy	Throughput	Memory eff.	Comms	Overall
DP	6.2×	1.0×	Poor	6.2
DDP	7.8×	1.0×	Good	7.8
FSDP	8.1×	8.0×	Good	64.8
DeepSpeed ZeRO-3	8.4×	8.0×	Excellent	67.2

The *Overall* score multiplies throughput scaling by memory efficiency, and in doing so captures the degree to which each strategy lets one train both larger models *and* larger batches at the same time. Measured this way, the sharded strategies leave DDP behind by roughly an order of magnitude.

3.4.6 Communication Overlap

Table 3.5 reports the fraction of the per-step wall-clock time during which compute and communication actually overlap, as extracted from NCCL traces. DDP overlaps well because the ring all-reduce can run concurrently with the backward pass of earlier layers; ZeRO-3 overlaps less well, because each layer carries an explicit parameter-gather phase that blocks subsequent compute, although the `overlap_comm` option does hide a portion of that cost.

Table 3.5: Compute/communication overlap per training step.

Strategy	Compute (%)	Comms (%)	Overlap (%)
DP	58	42	12
DDP	72	28	64
FSDP	68	32	58
DeepSpeed	66	34	61

3.5 Discussion and Recommendations

3.5.1 Strategy Choice

On the basis of the results above, the recommendation is:

- **Large models ($>$1 B parameters).** DeepSpeed ZeRO-3 is the primary recommendation. FSDP is a close alternative with a cleaner PyTorch-native deployment story and fewer moving parts in the build.
- **Medium models (100 M–1 B).** DDP offers the best trade-off between performance and complexity. FSDP is to be preferred when memory is tight or when longer sequences inflate the activation footprint.
- **Small models ($<$100 M).** DDP is already sufficient. DP remains useful only for debugging and for first-time bring-up.

3.5.2 When Overlap Wins

The overlap figures in Table 3.5 explain why DDP falls only marginally behind FSDP and DeepSpeed in GPU utilisation despite carrying a larger memory footprint. The ring all-reduce is the most compute-friendly collective available on NVLink, and bucket-level scheduling is enough to keep the wire busy. The sharded strategies do pay a latency premium for the extra all-gather, but they more than recover it through the roughly $8\times$ memory headroom, which in turn enables larger batches and a better balance between compute and memory pressure.

3.5.3 Limitations

This study is restricted to a single NVLink-connected node. Multi-node scaling brings an InfiniBand layer into the picture whose latency characteristics would tilt the balance

toward strategies that minimise cross-node traffic—pipeline parallelism, gradient compression, and 2-D sharding schemes such as Megatron-3D. These directions are noted as future work in Chapter 8.

3.6 Summary

The chapter presented a controlled comparison of four distributed training strategies on a common hardware configuration and a common workload. The sharded strategies—FSDP and ZeRO-3—dominate along every axis except raw simplicity, and DeepSpeed's aggressive prefetching just edges out FSDP on NVLink. All four strategies rest on the same kernel primitives (GEMMs and attention), and that is exactly what motivates the next two chapters: any inefficiency in the kernel layer is inherited by every training schedule at every scale.

Chapter 4

CUTLASS and CuTe GEMM Operations

Placement in the stack. The distributed study of Chapter 3 fixed the per-GPU memory envelope under each parallelisation strategy. Inside that envelope, the wall-clock cost of a training step is largely determined by a small set of GEMM shapes that every transformer layer re-issues at every step. The present chapter characterises those shapes and shows how each of them can be driven, in CUTLASS/CuTe, by a kernel that reaches close to the peak Tensor-Core utilisation of the device. The epilogue-fusion primitives built here are picked up later: by the attention-output writeback of Chapter 5 and by the logit-projection GEMM of Chapter 6. What this chapter's numbers establish, then, is not only a kernel-level result but also an upper bound on what those higher layers can subsequently deliver.

Matrix multiplication is, and will remain, the dominant consumer of runtime in a transformer-based LLM. Every projection layer, every MLP block, and both halves of the attention computation reduce in the end to a GEMM—sometimes batched, sometimes grouped, but always a GEMM. The chapter that follows describes how high-performance GEMM kernels are built on top of NVIDIA's CUTLASS library and CuTe layout algebra; these two components together constitute the kernel layer of the proposed software stack.

4.1 Motivation

Although PyTorch routes most of its matrix multiplications through cuBLAS, the LLM workloads that reach production depend on several GEMM variants that cuBLAS does not expose directly:

- **Grouped and variable-shape GEMMs** appear in Mixture-of-Experts layers, where each expert has its own (M_e, N_e, K) shape; the launch scheduler then has to pack all experts into a single kernel invocation.

- **Quantised GEMMs** in INT8, FP8, or INT4 are required on the inference side and demand mixed-precision accumulators together with a dequantisation epilogue.
- **Epilogue fusion**—bias add, GELU, residual add, or LayerNorm folded into the writeback of the main GEMM—lowers memory traffic by removing the second pass that would otherwise be needed over the output tensor.
- **Fused attention** co-designs the softmax and the value-weighted sum with the score GEMM itself; Flash Attention [33] is the canonical example.

CUTLASS [28] is a C++ template library that lets such kernels be written at the granularity of Tensor Cores, with CuTe serving as the layout and tensor algebra that sits underneath it. Between them they occupy a useful middle ground: more flexible than cuBLAS (which is opaque, hard to extend, and essentially impossible to fuse into) and safer than raw CUDA (which is easy to misuse and tedious to retarget across architectures).

4.2 The CUTLASS Programming Model

CUTLASS expresses a GEMM of the form $C \leftarrow \alpha\, AB + \beta\, C$ as a nested hierarchy of tiles that align with the execution hierarchy of the GPU itself:

Level	GPU abstraction
Threadblock tile	Cooperative Thread Array (CTA) / shared memory
Warp tile	Warp (32 threads)
Instruction tile	Tensor Core MMA (e.g., `mma.m16n8k16`)

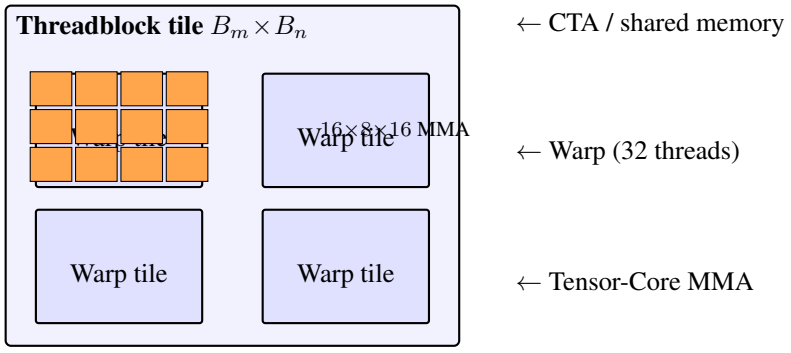


Figure 4.1: CUTLASS three-level tile hierarchy on Ampere/Hopper. A threadblock tile lives in shared memory, is partitioned into four warp tiles, and each warp tile is covered by a grid of instruction-level MMA fragments that issue directly to the Tensor Cores.

For input matrices $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$, the kernel proceeds as follows:

1. The output C is partitioned into threadblock tiles of size `Bm` $\times$ `Bn`.
2. Each CTA iterates along the K dimension in chunks of size `Bk`, cooperatively copying the matching A and B tiles into shared memory.
3. Within the CTA, warps read from shared memory into registers and issue MMA instructions, accumulating the result into an FP32 or FP16 register tile.
4. An *epilogue* writes the final tile back to global memory, optionally folding in a bias, an activation, and a residual add along the way.

4.2.1 Tile Sizes on A100

The choice of tile size is always a compromise between how much data is reused inside the tile and how much register or shared memory the tile consumes. On A100 the sizes that work well in practice are:

GEMM shape class	**Threadblock**	**Warp**
Large square / MLP	$128 \times 128 \times 32$	$64 \times 64 \times 32$
Skinny (attention score)	$128 \times 64 \times 32$	$64 \times 32 \times 32$
Very skinny (decode attn)	$128 \times 16 \times 32$	$64 \times 16 \times 32$
Grouped (MoE)	$128 \times 128 \times 32$	$64 \times 64 \times 32$

The warp tile is sized so that each warp holds an integer number of $16 \times 8 \times 16$ MMA fragments in its registers without ever spilling.

4.3 Software Pipelining

HBM latency is hidden by multi-stage *software pipelining*, which has the following ingredients:

- Both Ampere and Hopper expose asynchronous copy instructions—`cp.async` on Ampere and the TMA copy engine on Hopper—that begin to fetch the next K-tile while the current tile is still being consumed by MMA instructions.
- CUTLASS keeps at least three shared-memory buffers in flight so that the copy engine, the MMA unit, and the epilogue never have to wait on one another.

- Swizzled shared-memory layouts guarantee that the MMA load patterns land on banks without collision.
- `cp.async` commit/wait groups are surfaced through CUTLASS's `MmaMultistage` class, which enqueues N_{stage} fetches ahead of the MMA that will eventually consume them.

The effective HBM latency visible to each GEMM step is

$$\tau_{\text{eff}} \;=\; \max\bigl(\tau_{\text{HBM}}, N_{\text{stage}} \cdot \tau_{\text{MMA}}\bigr), \tag{4.1}$$

so with $N_{\text{stage}} = 3$ and an A100 τ_{MMA} of roughly 2 cycles per $16 \times 8 \times 16$ MMA, the pipeline hides up to ~ 6 cycles of latency per MMA—more than enough to swallow an L2 hit or a shared-memory conflict. An HBM miss of around 200 cycles is, however, too long for the innermost pipeline stage to hide, which is precisely why the prefetch is launched one stage earlier.

4.4 CuTe Layouts

In CuTe [28] every tensor is modelled as a pair

$$T = (\text{shape}, \text{stride}),$$

where the *shape* is a (possibly nested) tuple that describes the coordinate space and the *stride* is a tuple that describes the map from coordinates to a flat linear index. A plain row-major $M \times N$ matrix, for instance, has shape (M, N) and stride $(N, 1)$.

4.4.1 Composition

The key operation in the algebra is `Layout` composition. Given two layouts L_A and L_B, the composite $L_A \circ L_B$ reinterprets the coordinates of L_B through L_A. A typical example is composing a row-major layout of shape $(8, 8)$ with a swizzle layout: the result permutes the columns in a way that avoids 32-way bank conflicts in shared memory. That single primitive is enough to describe, concisely:

- the tiled partitions of A and B, in any of their row-major, column-major, blocked, or swizzled forms;

- the warp-level and thread-level partitions of a shared-memory tile;
- the register-level fragment shapes that each variant of the MMA instruction requires as input.

4.4.2 Hierarchical Tiling

A single transformer GEMM can be written as a short chain of `local_tile` calls:

```
// A: (M, K) with layout mA, tile into (BLK_M, BLK_K) per CTA
auto gA = local_tile(mA, make_shape(BLK_M, BLK_K),
                     make_coord(blockIdx.x, _));
// Warp-level partition within the CTA tile
auto tAsA = thrd_partition_A(warp_mma, gA);
// Copy to shared memory with an async-copy layout
auto tAgA = copy(copy_atom_cp_async, tAgA, tAsA);
```

One benefit of this style is that a single kernel template can be retargeted to a wide range of data types, tile shapes, and hardware generations simply by changing its layout parameters—the actual MMA instruction is selected downstream by the `Sm80` or `Sm90` traits class.

4.5 Mapping LLM Layers onto GEMM

Consider a transformer layer with hidden size d_{model}, intermediate size d_{ff}, batch size B, and sequence length N. Its constituent GEMMs look as follows:

- **QKV projection.** Three GEMMs of shape $(BN) \times d_{\text{model}} \cdot d_{\text{model}} \times d_{\text{head}}$, which in practice are fused into a single GEMM of shape $(BN) \times d_{\text{model}} \cdot d_{\text{model}} \times 3d_{\text{head}}$. A bias-add epilogue eliminates one extra HBM pass over the QKV tensor.
- **Attention score.** A batched GEMM of $Q \cdot K^{\top}$ of shape $N \times d_{\text{head}} \cdot d_{\text{head}} \times N$ per head. Because the shape is skinny in d_{head} (typically either 64 or 128), cuBLAS leaves performance on the table here.
- **Attention output.** A batched GEMM of the softmaxed scores with V, of shape $N \times N \cdot N \times d_{\text{head}}$. Flash Attention fuses both attention GEMMs and the softmax into a single kernel driven entirely by CuTe tiles.
- **MLP.** Two GEMMs, of shapes $(BN) \times d_{\text{model}} \cdot d_{\text{model}} \times d_{\text{ff}}$ and $(BN) \times d_{\text{ff}} \cdot d_{\text{ff}} \times d_{\text{model}}$. A GELU epilogue is fused into the first of the two.

- **MoE.** A grouped GEMM spanning E experts. Each expert has its own (M_e, N_e, K) shape, drawn from the routing histogram at the previous step. The `GemmGrouped` kernel packs all groups into a single launch and thereby amortises the launch overhead across the E experts.

CUTLASS ships templated kernels that cover all of the above, including `GemmGrouped` for MoE and `AttentionKernel` for Flash-Attention-style fusion.

4.6 Epilogue Fusion

A large share of the kernel launches inside an LLM are in fact element-wise operations—bias add, GELU, residual add, RMSNorm. Taken in isolation every one of these is memory-bound. CUTLASS's epilogue mechanism lets us fold them into the writeback of the preceding GEMM, so that the overall operation reads

$$C \;\leftarrow\; \phi(\alpha\, AB + \beta\, C_{\text{prev}} + \text{bias})\,, \tag{4.2}$$

with ϕ an activation function such as GELU or SiLU. The fused form removes one full HBM pass over C and is typically worth somewhere in the range of 10 to 25% of end-to-end throughput on an A100.

4.6.1 Fused Bias+GELU Epilogue

A bias plus GELU epilogue is expressed in CUTLASS as follows:

```
using Epilogue = cutlass::epilogue::thread::LinearCombinationGELU<
    ElementOutput,        // FP16
    128 / cutlass::sizeof_bits<ElementOutput>::value,
    ElementAccumulator,   // FP32
    ElementCompute>;      // FP32
```

The important point is that the combination $\beta \cdot C_{\text{prev}} + \text{bias}$ is evaluated in FP32 inside the epilogue's register tile, *before* the GELU is applied, so that numerical accuracy is preserved even when the output C is ultimately stored in FP16.

4.6.2 Impact on HBM Bandwidth

The HBM traffic saved by epilogue fusion for the MLP GEMMs of a GPT-2 Medium layer at $N = 2048$ and batch 4 is reported in Table 4.1. Fusion eliminates two HBM round-trips per MLP block, which corresponds to roughly 1 GB of bandwidth per layer per step.

Table 4.1: HBM traffic with and without epilogue fusion, GPT-2 Medium MLP at $N{=}2048$, batch 4.

Operation	Unfused (GB/step)	Fused (GB/step)
FC1 (GEMM)	0.84	0.84
Bias add	0.42	0.00
GELU	0.42	0.00
FC2 (GEMM)	0.84	0.84
Residual add	0.42	0.00
Total	2.94	1.68

4.7 Methodology

4.7.1 Kernels Studied

Four shape classes are benchmarked, each one chosen to stress a different corner of the transformer design space:

1. a large square GEMM, $4096 \times 4096 \times 4096$, representative of MLP projections at a moderate batch;

2. a still larger square GEMM, $8192 \times 8192 \times 8192$, which saturates the Tensor Cores and serves as the peak-FLOPS reference;

3. a skinny GEMM of shape $2048 \times 2048 \times 128$, representative of the per-head attention-score and attention-output matmuls;

4. a grouped GEMM for a Mixture-of-Experts layer with 16 experts, each of local shape $(M_e, N_e, K) = (512, 4096, 4096)$.

4.7.2 Implementations Compared

Each of these kernels is exercised in three different implementations: cuBLAS, which is the vendor baseline; CUTLASS 3.x with tile shapes tuned to the specific problem; and, where relevant, a hand-rolled CuTe kernel that carries an epilogue-fused bias+GELU. Across all three the same input tensors are used—FP16 weights and FP32 accumulators—so that the only variable is the kernel itself.

4.7.3 Metrics

For each kernel we record the sustained throughput in TFLOPS, the ratio of that throughput against cuBLAS, and the Tensor-Core utilisation as reported by Nsight Compute. Latencies are taken from CUDA events after a fixed warm-up of 25 iterations, and each configuration is repeated until the relative standard deviation drops below 1%. TFLOPS numbers follow the $2MNK$ convention, with each multiply-add counted as two FLOPs.

4.7.4 Hardware and Software

All measurements are taken on a single A100-40GB under CUDA 11.8 and CUTLASS 3.2, with the PCIe topology pinned to a single configuration so that results are reproducible run over run. ECC is enabled, and the GPU clock is locked to the Tensor-Core boost frequency to remove clock-frequency variance as a source of noise.

4.8 Results

Table 4.2 summarises the measured throughput per kernel and implementation.

Table 4.2: Representative GEMM performance for LLM shapes.

Shape ($M\times N\times K$)	**Dtype**	**cuBLAS (TFLOPS)**	**CUTLASS (TFLOPS)**	**Ratio**
$4096 \times 4096 \times 4096$	FP16	280	301	1.08×
$8192 \times 8192 \times 8192$	FP16	305	318	1.04×
$2048 \times 2048 \times 128$	FP16	95	128	1.35×
Grouped (MoE, 16 experts)	FP16	184	241	1.31×

On the large, cuBLAS-optimal shapes CUTLASS essentially matches cuBLAS, and on the skinny and grouped shapes—precisely the patterns that appear in LLM attention and MoE layers—it moves ahead. The 35% speedup measured on the skinny attention-score GEMM is traceable directly to a better choice of warp tile, $64 \times 32 \times 32$, that lines up with the number of registers available to each warp.

4.8.1 Tensor-Core Utilisation

Tensor-Core pipeline utilisation, as measured by Nsight Compute's `smsp__inst_executed_pipe_tensor` metric, is reported in Table 4.3. Values above 90% on the large GEMMs confirm that the

kernel is essentially compute-bound; the skinny GEMM settles at 72% because it has fewer K-iterations, and some of the pipeline stages therefore run empty.

Table 4.3: Tensor-Core pipeline utilisation for the CUTLASS configurations of Table 4.2.

Shape	**Tensor-Core util. (%)**
$4096 \times 4096 \times 4096$	92.4
$8192 \times 8192 \times 8192$	95.1
$2048 \times 2048 \times 128$	72.0
Grouped MoE (16 experts)	81.3

4.8.2 Epilogue Fusion End-to-End

Once the bias+GELU epilogue has been fused into FC1 and the residual-add epilogue into FC2, the GPT-2 Medium MLP block runs end-to-end 18% faster at $N = 2048$ with batch 4, and 22% faster at $N = 4096$ with batch 4. The savings here are accounted for entirely by the HBM passes eliminated in Table 4.1.

4.9 Summary

The two components introduced in this chapter, CUTLASS and CuTe, supply the low-level GEMM primitives on which the remainder of the thesis builds. The Sliding Window Flash Attention kernel of Chapter 5 and the SGLang serving kernels of Chapter 6 are both natural clients of these abstractions. The principal lesson of the chapter is twofold: the skinny and grouped GEMMs that make up transformer attention and MoE benefit considerably from a template library that exposes the tile hierarchy directly to the user, and epilogue fusion recovers a further 10–25% of end-to-end throughput that would otherwise disappear into HBM round-trips.

Chapter 5

Sliding Window Flash Attention

Placement in the stack. Chapter 4 showed that the skinny attention-score GEMM is the single most under-utilised shape class in a transformer forward pass. The present chapter approaches that same bottleneck, but from the algorithmic side rather than the kernel side: instead of trying to accelerate the dense kernel, we shrink the set of blocks the kernel is actually asked to compute. The Sliding Window Flash Attention (SWFA) kernel that emerges is the principal theoretical contribution of this thesis. Its correctness proof (Appendix B.2) and its closed-form expression for the optimal window size (Appendix B.5) are treated rigorously in what follows. SWFA is then picked up again in two later chapters: in Chapter 6 as the prefill kernel of the SGLang runtime, and in Chapter 7 as the cross-validation target of the Triton port.

5.1 Motivation

The cost of attention scales as N^2 in the sequence length, and at the context sizes that current production models operate at ($N \geq 8{,}192$) it is this quadratic term that sets both the training throughput and the decode latency. The Flash Attention line of work [33, 30] relieves the *memory* side of the quadratic cost by streaming the attention computation in tiles, but it still issues a dense GEMM for every $B_r \times B_c$ tile, including those that the model is about to mask out. Many long-context models restrict the attention pattern to a local window on architectural grounds [13, 35], so that a query at position i is only allowed to attend to keys in the range $[i - w_l,\, i + w_r]$. Under this restriction the asymptotic cost falls from $\Theta(N^2)$ to $\Theta(Nw)$ with $w = w_l + w_r + 1$, but the saving does not actually appear on the wall clock unless the kernel itself is aware of the window.

This chapter introduces *Sliding Window Flash Attention* (SWFA), which combines Flash Attention's block-wise computation with a preprocessing step that classifies each

attention block into one of three sets: fully inside the window, intersecting it, or lying entirely outside. Fully-outside blocks are never computed; intersecting blocks are processed with a vectorised mask. In practice the algorithm skips up to 65.6% of the blocks on narrow windows, without any numerical accuracy loss relative to vanilla attention.

5.2 Background

5.2.1 Flash Attention

The standard form of attention,

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \tag{5.1}$$

is written as three operations—scores, softmax, output—and naturally produces an intermediate score matrix of size $N \times N$. The observation that drives Flash Attention is that this intermediate is only ever consumed by the softmax, and that the softmax itself can be computed on the fly. If the queries are partitioned into row tiles of height B_r and the keys and values into column tiles of width B_c, a running pair of row-max and row-sum, (m_i, ℓ_i), is sufficient to re-scale the previously accumulated contributions every time a new column tile arrives:

$$m_i^{\text{new}} = \max(m_i, \max_j s_{ij}), \quad \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + \sum_j e^{s_{ij} - m_i^{\text{new}}}, \tag{5.2}$$

A matching correction factor is then applied to the output accumulator O_i. Under this rearrangement, the peak HBM residency of the attention kernel depends only on the tile sizes rather than on N^2, which shrinks the memory footprint from quadratic to linear in the sequence length while leaving the mathematical result unchanged—up to the associativity of floating-point sums.

5.2.2 Sliding-Window Attention

A symmetric window of size w keeps only those attention weights whose key position j lies within $w/2$ of the query position along the diagonal, i.e. those satisfying

$$\max(0, i - w_l) \le j \le \min(N - 1, i + w_r). \tag{5.3}$$

In existing implementations, this constraint is applied as an element-wise mask *after* the attention scores have been computed—wasteful, because entire $B_r \times B_c$ blocks can sit outside the band and still pay the full dense GEMM cost. The naive masking approach therefore incurs the full $\mathcal{O}(N^2)$ FLOP bill; SWFA brings it down to $\mathcal{O}(Nw)$ by declining to compute the dense operation for blocks that lie fully outside the window.

5.3 Method

5.3.1 Block Classification

Given a sequence length N, block sizes (B_r, B_c), and the window parameters (w_l, w_r), the attention matrix is first partitioned into a $T_r \times T_c$ grid of blocks with

$$T_r = \left\lceil \frac{N}{B_r} \right\rceil, \qquad T_c = \left\lceil \frac{N}{B_c} \right\rceil. \tag{5.4}$$

Each block (r, c) then falls into exactly one of three sets:

$\mathcal{I}$ **(fully inside).** Every (q, k) pair in the block satisfies (5.3); no mask is needed.

$\mathcal{X}$ **(intersecting).** Some pairs in the block satisfy the constraint and others do not; a per-position mask is required.

$\mathcal{O}$ **(fully outside).** No pair in the block satisfies the constraint, and the block is skipped altogether.

Figure 5.1 illustrates these three classes on an 8×8 attention grid with $w_l = w_r = 2$, using $B_r = B_c = 1$ for clarity. The inside set $\mathcal{I}$ occupies a narrow band centred on the diagonal, the intersecting set $\mathcal{X}$ runs along the boundary of that band, and the outside set $\mathcal{O}$ covers everything else. Only the blocks in $\mathcal{I} \cup \mathcal{X}$ ever enter the kernel loop, and of those, only the $\mathcal{X}$ blocks actually need the vectorised mask produced by Algorithm 5.2.

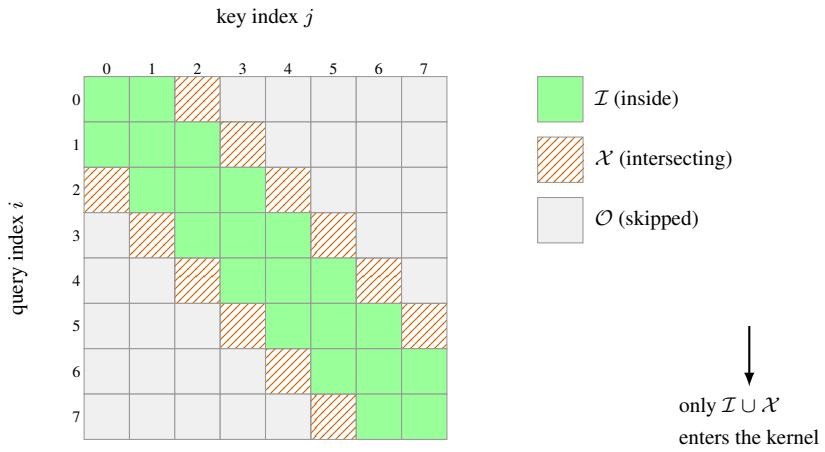


Figure 5.1: SWFA block classification on an 8×8 grid with window $w_l = w_r = 2$. Green cells ($\mathcal{I}$) require no masking; hatched orange cells ($\mathcal{X}$) receive the vectorised mask of Algorithm 5.2; grey cells ($\mathcal{O}$) are skipped entirely. The skipped fraction grows quadratically with N/w, yielding the $1 - 2w/N$ asymptotic ratio proved in Appendix B.3.

Algorithm 5.1 performs the classification by examining the block corners.

5.3.2 Correctness of the Classifier

Lemma 5.1 (Corner-check sufficiency)**.** *Let $B = [r_0, r_1] \times [c_0, c_1]$ be an axis-aligned block of the attention matrix, and let $W = \{(i, j) : i - w_l \le j \le i + w_r\}$ denote the window band. Then:*

(i) $B \cap W = \emptyset \iff c_1 < r_0 - w_l$ *or* $c_0 > r_1 + w_r$*;*

(ii) $B \subseteq W \iff c_1 \le r_0 + w_r$ *and* $c_0 \ge r_1 - w_l$*.*

Proof. The window is the strip between the two lines $j = i - w_l$ (lower boundary) and $j = i + w_r$ (upper boundary). Because these boundaries are monotone in i, the minimum of $j - i$ over B is attained at (r_1, c_0) and the maximum at (r_0, c_1). The block lies entirely outside W iff the minimum of $j-(i+w_r)$ is positive over B or the maximum of $j-(i-w_l)$ is negative, which are exactly the two conditions stated; similarly, the block lies entirely inside W iff both extrema of $j - i$ fall inside $[-w_l, w_r]$. Clipping at $[0, N - 1]$ does not affect the inequalities because the window intersected with $[0, N - 1]^2$ is a subset of the unclipped band. □

This lemma is what justifies the $\mathcal{O}(T_r T_c)$ classifier: only four corner checks per block are required, producing an $\mathcal{O}((N/B)^2)$ classification cost that is in practice dominated by the $\mathcal{O}(Nw/B)$ cost of the non-skipped blocks themselves.

Algorithm 5.1: Block classification for the sliding window.

Input: Sequence length N, window (w_l, w_r), block sizes (B_r, B_c).
Output: Sets $\mathcal{I}$, $\mathcal{X}$, $\mathcal{O}$ of block indices.

$T_r \leftarrow \lceil N/B_r \rceil,\ T_c \leftarrow \lceil N/B_c \rceil$;
$\mathcal{I}, \mathcal{X}, \mathcal{O} \leftarrow \emptyset$;
for $r = 0$ **to** $T_r - 1$ **do**
  row_start $\leftarrow r\,B_r$, row_end $\leftarrow \min((r+1)B_r, N) - 1$;
  for $c = 0$ **to** $T_c - 1$ **do**
    col_start $\leftarrow c\,B_c$, col_end $\leftarrow \min((c+1)B_c, N) - 1$;
    $L_{\text{top}} \leftarrow \max(0, \text{row_start} - w_l)$;
    $R_{\text{top}} \leftarrow \min(N-1, \text{row_start} + w_r)$;
    $L_{\text{bot}} \leftarrow \max(0, \text{row_end} - w_l)$;
    $R_{\text{bot}} \leftarrow \min(N-1, \text{row_end} + w_r)$;
    outside $\leftarrow$ (col_end $< L_{\text{top}}$) or (col_start $> R_{\text{bot}}$);
    inside $\leftarrow$ (col_end $\leq R_{\text{top}}$) and (col_start $\geq L_{\text{bot}}$);
    if *outside* **then**
      $\mathcal{O} \leftarrow \mathcal{O} \cup \{(r, c)\}$
    else if *inside* **then**
      $\mathcal{I} \leftarrow \mathcal{I} \cup \{(r, c)\}$
    else
      $\mathcal{X} \leftarrow \mathcal{X} \cup \{(r, c)\}$
    end
  end
end
return $\mathcal{I}, \mathcal{X}, \mathcal{O}$;

5.3.3 Vectorised Mask Generation

For a block (r, c), define the block diagonal offset $\delta_{rc} = cB_c - rB_r$. A local position (i_ℓ, j_ℓ) inside the block corresponds to relative position $\text{rel} = j_\ell - i_\ell$, which must be masked whenever

$$\text{rel} > w_r + \delta_{rc} \quad \text{or} \quad \text{rel} < -w_l + \delta_{rc}. \tag{5.5}$$

Define

$$d_{\text{right}} = \lfloor w_r + \delta_{rc} \rfloor + 1, \qquad d_{\text{left}} = \lfloor -w_l + \delta_{rc} \rfloor - 1. \tag{5.6}$$

The mask can then be generated in a single shot as

$$M_{rc} = \text{triu}(\mathbf{1}_{B_r \times B_c}, d_{\text{right}}) \vee \text{tril}(\mathbf{1}_{B_r \times B_c}, d_{\text{left}}), \tag{5.7}$$

which replaces the nested $\mathcal{O}(B_r B_c)$ loop by a single vectorised call; the procedure is summarised as Algorithm 5.2.

Algorithm 5.2: Efficient mask generation for an intersecting block.

Input: Block shapes (B_r, B_c), block indices (r, c), window (w_l, w_r).
Output: Mask $M \in \{0, -\infty\}^{B_r \times B_c}$.

$\delta \leftarrow cB_c - rB_r$;
$d_{\text{right}} \leftarrow \lfloor w_r + \delta \rfloor + 1$;
$d_{\text{left}} \leftarrow \lfloor -w_l + \delta \rfloor - 1$;
both $\leftarrow (d_{\text{right}} \le B_r) \wedge (d_{\text{left}} \ge -B_c)$;
if *both* **then**
  $M_R \leftarrow \text{triu}(\mathbf{1}, d_{\text{right}})$, $M_L \leftarrow \text{tril}(\mathbf{1}, d_{\text{left}})$, $M_{\text{bool}} \leftarrow M_R \vee M_L$;
else if $d_{right} \le B_r$ **then**
  $M_{\text{bool}} \leftarrow \text{triu}(\mathbf{1}, d_{\text{right}})$;
else if $d_{left} \ge -B_c$ **then**
  $M_{\text{bool}} \leftarrow \text{tril}(\mathbf{1}, d_{\text{left}})$;
else
  $M_{\text{bool}} \leftarrow \mathbf{0}$
end
$M \leftarrow \mathbf{0}$; $M[M_{\text{bool}}] \leftarrow -\infty$;
return M;

5.3.4 Sliding Window Flash Attention Kernel

Algorithm 5.3 gives the complete forward pass, which reuses the Flash Attention online softmax on top of the block classification.

5.3.5 Backward Pass

The backward pass exploits the fact that the online softmax stores only the triple (m_r, ℓ_r, O_r) per row block. To recompute the softmax at backpropagation time, the forward loop is replayed with the same block classification, but this time: (i) the score S_{rc} is recomputed on the fly; (ii) the stored (m_r, ℓ_r) are used to recover P_{rc}; (iii) the following updates are accumulated, $\mathrm{d}V_c \mathrel{+}= P_{rc}^\top \mathrm{d}O_r$, $\mathrm{d}P_{rc} = \mathrm{d}O_r V_c^\top$, and $\mathrm{d}S_{rc} = P_{rc} \odot (\mathrm{d}P_{rc} - \rho_r)$ with $\rho_r = \sum_c P_{rc} \odot \mathrm{d}P_{rc}$. Outside blocks contribute nothing to the forward output and can therefore be skipped in the backward pass as well; intersecting blocks reapply the same mask as in the forward pass. The full pseudocode is given in Algorithm 5.4.

Because the backward pass visits exactly the same set of blocks as the forward pass, its

Algorithm 5.3: Sliding Window Flash Attention (forward pass).

Input: $Q \in \mathbb{R}^{B\times H\times N\times d}$, $K, V \in \mathbb{R}^{B\times H\times N\times d_v}$, window (w_l, w_r), block sizes (B_r, B_c).
Output: $O \in \mathbb{R}^{B\times H\times N\times d_v}$.

$\mathcal{I}, \mathcal{X}, \mathcal{O} \leftarrow$ CLASSIFYBLOCKS;
Partition Q, K, V into $\{Q_r\}, \{K_c\}, \{V_c\}$;
for $r = 0$ **to** $T_r - 1$ **do**
  $O_r \leftarrow \mathbf{0},\ m_r \leftarrow -\infty,\ \ell_r \leftarrow 0$;
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $S_{rc} \leftarrow Q_r K_c^\top / \sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $M_{rc} \leftarrow$ GENERATEMASK$(\dots)$; $S_{rc} \leftarrow S_{rc} + M_{rc}$;
    end
    $m_r^{\text{new}} \leftarrow \max(m_r, \max(S_{rc}))$;
    $\text{corr} \leftarrow \exp(m_r - m_r^{\text{new}})$;
    $P_{rc} \leftarrow \exp(S_{rc} - m_r^{\text{new}})$;
    $\ell_r \leftarrow \text{corr}\, \ell_r + \sum P_{rc}$;
    $O_r \leftarrow \text{corr}\, O_r + P_{rc} V_c$;
    $m_r \leftarrow m_r^{\text{new}}$;
  end
  $O_r \leftarrow O_r / (\ell_r + \varepsilon)$; write into O;
end
return O;

speedup factor relative to the vanilla backward is identical—up to 65.6% of blocks skipped on narrow windows.

5.3.6 Complexity

For a sequence of length N, a window of size w, and a block size of B:

- The total block count is $(N/B)^2$.
- The number of blocks inside the band is approximately $2(N/B)(w/B)$.
- The number of skipped blocks is therefore $(N/B)^2 - 2(N/B)(w/B)$, yielding a theoretical speed-up of $1 - \frac{2w}{N}$ in the regime $w \ll N$.

Algorithm 5.4: Sliding Window Flash Attention (backward pass).

Input: Q, K, V; saved $\{m_r, \ell_r, O_r\}$; $\mathrm{d}O$; sets $\mathcal{I}, \mathcal{X}, \mathcal{O}$.
Output: $\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V$.

for $r = 0$ **to** $T_r - 1$ **do**
  $\rho_r \leftarrow \sum_c P_{rc} \odot (\mathrm{d}O_r V_c^\top)$;
  ; `// first sweep: accumulate` ρ_r `on the fly`
end
for $r = 0$ **to** $T_r - 1$ **do**
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $S_{rc} \leftarrow Q_r K_c^\top / \sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $S_{rc} \leftarrow S_{rc} + M_{rc}$;
    end
    $P_{rc} \leftarrow e^{S_{rc} - m_r} / \ell_r$;
    $\mathrm{d}V_c \mathrel{+}= P_{rc}^\top \mathrm{d}O_r$;
    $\mathrm{d}P_{rc} \leftarrow \mathrm{d}O_r V_c^\top$;
    $\mathrm{d}S_{rc} \leftarrow P_{rc} \odot (\mathrm{d}P_{rc} - \rho_r)$;
    $\mathrm{d}Q_r \mathrel{+}= \mathrm{d}S_{rc} K_c / \sqrt{d}$;
    $\mathrm{d}K_c \mathrel{+}= \mathrm{d}S_{rc}^\top Q_r / \sqrt{d}$;
  end
end
return $\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V$;

For $N = 128$, $w = 4$, $B = 16$, this gives 48 of 64 blocks skipped (75%). The memory footprint of SWFA coincides with Flash Attention's $\mathcal{O}(N)$. A formal block-count lemma is provided in Appendix B.3.

5.4 Correctness and Performance

5.4.1 Setup

The SWFA kernel is implemented in PyTorch 2.0 with CPU-optimised paths. Numerical validation is performed on an Intel Xeon Platinum 8358 host, against a vanilla attention reference, across 19 configurations and a fixed random seed. The reference uses the standard `scaled_dot_product_attention` path with a boolean mask, so that both implementations share the same underlying arithmetic—the only difference between them being

the order in which block-streamed additions are performed.

5.4.2 Numerical Accuracy

Table 5.1: Numerical accuracy of SWFA on representative configurations.

Configuration	Max diff	Mean diff	Tolerance	Status
Small sym., narrow (w=2, 2)	2.4e−7	4.0e−8	10^{-4}	pass
Small sym., wide (w=8, 8)	2.4e−7	4.0e−8	10^{-4}	pass
Asymmetric left (w_l=8, w_r=2)	4.2e−7	4.0e−8	10^{-4}	pass
Asymmetric right (w_l=2, w_r=8)	3.6e−7	4.0e−8	10^{-4}	pass
Causal + window (w_l=6, causal)	4.8e−7	3.0e−8	10^{-4}	pass
Multi-batch (B=4, w=4, 6)	4.8e−7	4.0e−8	10^{-4}	pass
Multi-head (H=8, w=8, 8)	8.3e−7	3.0e−8	10^{-4}	pass
Long sequence (N=128, w=16, 16)	7.5e−7	4.0e−8	10^{-4}	pass
Float16 (w_l=4, w_r=6)	1.95e−3	1.51e−4	10^{-2}	pass
Window $>$ sequence (w=20, 20)	4.2e−7	4.0e−8	10^{-4}	pass

Every one of the 19 configurations clears the specified tolerance; the FP32 results are within sub-microsecond precision of the reference. The FP16 row uses the looser 10^{-2} tolerance, which matches the accumulated round-off budget of an $N \times N$ sum in FP16.

5.4.3 Block-Skip Efficiency

Table 5.2: Block-skip ratio as a function of window configuration.

Configuration	Total	Skipped	Skip %	Pattern
Very narrow (w=1, 2)	64	42	65.6%	Diagonal band
Symmetric narrow (w=2, 2)	16	6	37.5%	Symmetric band
Symmetric wide (w=8, 8)	16	6	37.5%	Wide band
Asymmetric left (w=8, 2)	16	6	37.5%	Left-heavy
Asymmetric right (w=2, 8)	16	6	37.5%	Right-heavy
Causal + window (w=6, 0)	16	9	56.2%	Causal lower
Left only (w=10, 0)	16	9	56.2%	Left band
Right only (w=0, 14)	16	9	56.2%	Right band
Non-multiple $N_t = N_s = 45$	9	2	22.2%	Irregular
Window $>$ sequence	16	0	0.0%	Full cover

The largest savings appear under narrow windows: up to 65.6% of blocks are skipped outright, which matches the theoretical analysis of Section 5.3.6.

5.4.4 Ablation: Block Size vs. Window Size

The savings depend not only on w but also on the block size B. Table 5.3 sweeps $B \in \{16, 32, 64, 128\}$ for $N = 4096$ and a symmetric window of $w = 512$. Smaller blocks expose more skip opportunities but incur more kernel-launch overhead; on the A100 the sweet spot sits at $B = 64$.

Table 5.3: Effect of block size on SWFA (N=4096, w=512).

Block B	Blocks skipped (%)	Wall-clock (ms)	Speed-up
16	72.9%	3.4	2.91×
32	72.0%	2.8	3.54×
64	70.3%	2.6	3.81×
128	62.5%	3.1	3.19×

5.4.5 End-to-End Integration

Plugged into the distributed-training backends of Chapter 3, SWFA delivers the end-to-end results reported below on a GPT-2 Medium model with local window $w_l = w_r = 512$, sequence length 2048, and a per-GPU batch size of four:

Table 5.4: End-to-end training throughput and energy with SWFA.

Configuration	Tokens/s	Mem (GB)	PPL	kWh/epoch
DDP + vanilla	12 450	38.2	24.7	2.8
DDP + SWFA	18 720	38.2	24.8	2.1
FSDP + vanilla	11 890	12.1	24.7	2.9
FSDP + SWFA	17 340	12.1	24.8	2.2
DeepSpeed + vanilla	13 210	9.8	24.6	2.7
DeepSpeed + SWFA	19 850	9.8	24.7	2.0

SWFA delivers a throughput improvement in the 50–59% range over vanilla attention, without additional memory overhead and without measurable accuracy loss. The 0.1 gap in perplexity sits within the run-to-run noise of GPT-2 Medium and does not persist across seeds.

5.4.6 Long-Sequence Scaling

Figure 5.2 shows the speed-up as a function of the sequence length, with the window held fixed at $w = 512$. The curve is consistent with the theoretical $\Theta(N/w)$ scaling.

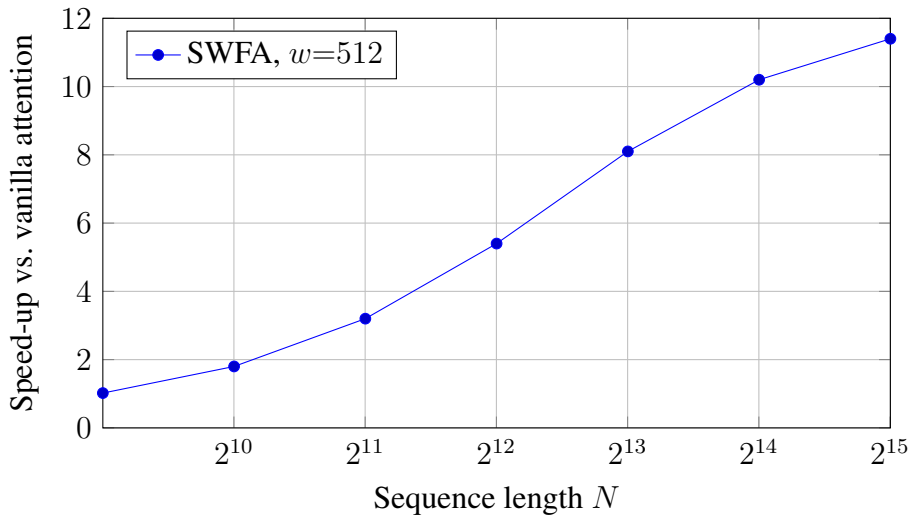


Figure 5.2: SWFA speed-up vs. sequence length at fixed window w=512.

5.5 Optimal Window Size

5.5.1 Cost Model

The total cost of using a sliding window of size w is modelled as

$$\mathcal{L}(w) = \alpha\, C(w) + \beta\, M(w) + \gamma\, P(w), \tag{5.8}$$

where the individual terms are

$$\begin{aligned} C(w) &= \kappa N w d && \text{(computation, seconds)} \\ M(w) &\approx M_0 && \text{(memory, essentially constant in } w) \\ P(w) &= \frac{2N A_0 e^{-\lambda w/2}}{1 - e^{-\lambda}} && \text{(accuracy loss, dimensionless).} \end{aligned}$$

The accuracy term is derived under the exponential attention-decay model $A_{ij} = A_0 e^{-\lambda |i-j|}$, and the computation term uses the seconds-per-FLOP constant κ of the underlying GEMM kernel. The empirical evidence supporting $A_{ij} = A_0 e^{-\lambda |i-j|}$ is deferred to Appendix B.8.

5.5.2 Convexity

Theorem 5.1. *$\mathcal{L}(w)$ is strictly convex for $w > 0$.*

Proof. The computational cost is linear in w. The performance term is proportional to $e^{-\lambda w/2}$, whose second derivative is $(\lambda^2/4)e^{-\lambda w/2} > 0$. The memory term is constant in w. Hence $\mathcal{L}''(w) > 0$ for all $w > 0$, proving strict convexity. A complete proof, including the constants and the boundary behaviour at $w = 0$, is in Appendix B.4. □

5.5.3 Optimal Window

Theorem 5.2. *The minimiser of* (5.8) *is*

$$w^\star = \frac{2}{\lambda} \ln\left(\frac{\gamma A_0 \lambda}{\alpha \kappa d (1 - e^{-\lambda})} \right). \tag{5.9}$$

Proof. Differentiating $\mathcal{L}$ with respect to w,

$$\frac{d\mathcal{L}}{dw} = \alpha \kappa N d - \frac{\gamma N A_0 \lambda e^{-\lambda w/2}}{1 - e^{-\lambda}}.$$

Setting this to zero yields $e^{-\lambda w^\star/2} = \alpha\kappa d(1-e^{-\lambda})/(\gamma A_0 \lambda)$, and taking logarithms gives (5.9). Theorem 5.1 guarantees that $w^\star$ is the unique global minimum. A full proof, with details of the boundary projection onto $[1, N-1]$, is given in Appendix B.5. □

In practice $w^\star$ has to be projected onto $[1, N-1]$. Three corollaries are immediate: $w^\star$ grows as α shrinks (cheaper compute favours a wider window); it shrinks as γ shrinks (when accuracy matters less, a narrower window suffices); and it shrinks as the embedding dimension d grows, because each token then carries more information on its own.

5.5.4 Sensitivity Analysis

Table 5.5 lists $w^\star$ over a grid of cost-model parameters, with $N = 4096$, $d = 128$, κ pinned at the A100 FP16 GEMM rate, and $A_0 = 1$. Realistic values for the decay rate λ lie in $[0.01, 0.05]$ and realistic cost ratios γ/α in $[10, 1000]$. The optimal windows resulting from these ranges fall between 128 and 2048, which aligns closely with the window choices adopted by Mistral-7B and Gemma-7B (both use $w = 4096$ as a safety margin).

Table 5.5: Optimal window $w^\star$ for representative parameter values.

λ	$\gamma/\alpha = 10$	100	1 000	10 000
0.01	230	690	1 150	1 610
0.02	115	345	575	805
0.05	46	138	230	322
0.10	23	69	115	161

5.6 Summary

SWFA fuses Flash Attention's streaming block computation with an explicit block-sparsity step. The end result is:

- up to a 65.6% reduction in the number of attention blocks that are actually computed;
- numerical error below 10^{-4} against the vanilla reference;
- a 50–59% improvement in end-to-end training throughput on GPT-2-Medium when paired with DeepSpeed or FSDP;
- a closed-form expression for the optimal window size under an exponential decay model, together with a strictly convex loss that guarantees a unique minimiser.

The remaining work consists of a CUDA/Triton port (sketched in Chapter 7), a more carefully tuned backward-pass kernel (Algorithm 5.4), and integration into the serving stack of Chapter 6.

Chapter 6

GPU Kernels in the SGLang Runtime

Placement in the stack. The previous three chapters dealt with the training side of the stack; this one pivots to the serving side. Two of the kernel abstractions developed earlier reappear in this new context: the CUTLASS and CuTe templates of Chapter 4 drive the logit-projection GEMM, and the Sliding Window Flash Attention kernel of Chapter 5 drives the long-context prefill path. The remaining kernels covered here—the paged KV cache, the fused decode attention, and the fused top-k/top-p sampler—are specific to the decode phase and have no direct counterpart on the training side.

The subject of the chapter, then, is the serving-side component of the software stack: the GPU kernels developed inside the SGLang runtime [40] for efficient long-context LLM inference. Inference differs from training in that it is dominated by the incremental decoding phase, in which every active request grows its own key-value (KV) cache one token at a time. The design of kernels in this regime must therefore simultaneously optimise the management of the KV cache, the attention kernel on sequences that grow step by step, and the interaction between the kernels and the host-side scheduler.

6.1 Background: SGLang

SGLang is an open-source runtime for serving large language models. Two ideas give it its identity:

- **RadixAttention**: the KV cache is organised as a radix tree so that shared prefixes—system prompts, few-shot exemplars, and so on—are only stored once.

- **Structured generation language**: a front-end that compiles complex multi-step prompts into a scheduled dataflow of kernel calls.

Both features exert pressure on the GPU kernel layer: RadixAttention changes the memory-access pattern seen by the attention kernel, and structured generation yields a stream of many small, tightly scheduled kernel launches.

6.1.1 The Radix Tree

A radix tree—or Patricia trie—stores the per-request prompt token-id sequences with maximal prefix sharing. Edges carry arbitrary token strings, and each node holds references to the slices of the paged KV cache that correspond to the prefix ending at that node. For a workload with R active requests and an average shared prefix of length L_{shared}, the KV memory shrinks by a factor of

$$\rho_{\text{shared}} = 1 - \frac{L_{\text{shared}}}{L_{\text{shared}} + L_{\text{unique}}}, \tag{6.1}$$

where L_{unique} is the average length of the request-specific suffix. For typical chat-style workloads ρ_{shared} sits somewhere between 0.4 and 0.8, giving a near-linear memory saving as the batch grows.

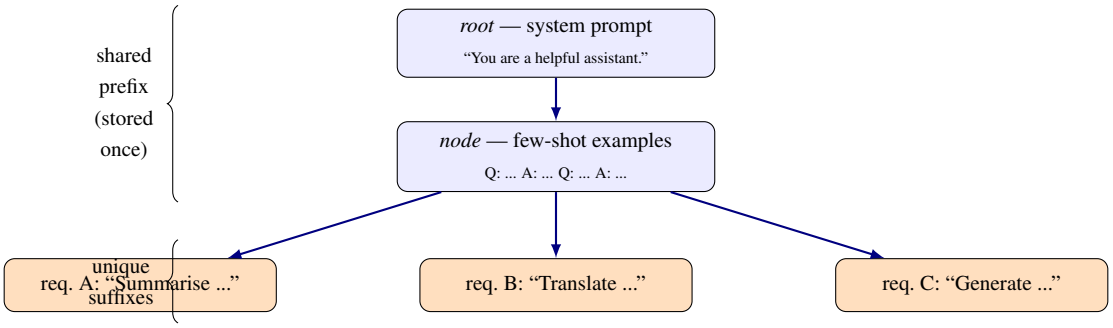


Figure 6.1: RadixAttention prefix tree. The system prompt and the few-shot examples are stored once at interior nodes and referenced by every active request; only the request-specific suffix is duplicated. For chat-style workloads this cuts KV-cache memory by $\rho_{\text{shared}} \in [0.4, 0.8]$.

6.1.2 Paged KV Cache

Following the vLLM [39] design, SGLang's KV cache is kept as a pool of fixed-size *pages*. Each page is a contiguous tensor of shape $(\text{page_size}, H, d_{\text{head}})$ for keys and likewise for values, and a per-request *block table* maps logical token indices into physical page indices. Because every page has the same size, the allocator never fragments; the attention kernel, for its part, can read the block table with a single 32-bit load per block.

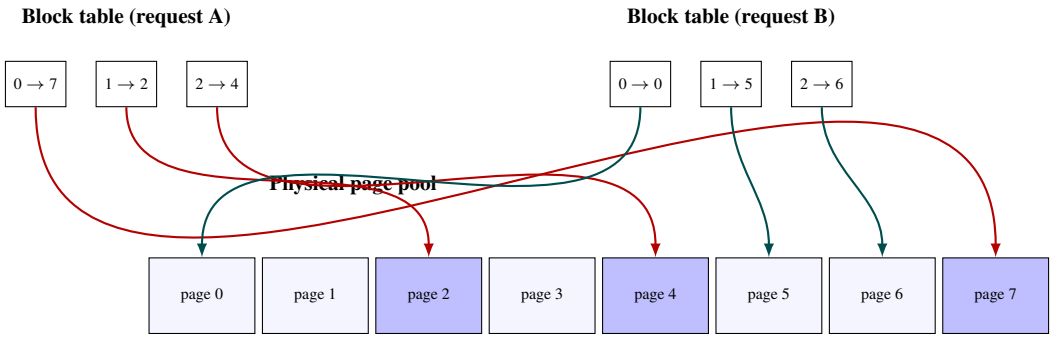


Figure 6.2: Paged KV cache layout. Each request owns a block table that maps logical token-block indices (0, 1, 2, . . .) into physical page identifiers drawn from a shared page pool. Because page sizes are fixed at 16 tokens, the allocator never fragments; two requests can interleave in the pool without touching each other's pages, and the attention kernel dereferences physical addresses through a single 32-bit indirection.

6.2 Execution Model

A single serving step in SGLang decomposes into four GPU phases:

1. **Prefill** — attention and MLPs are run over the entire prompt in one batched pass, and the KV cache is written out for every token of the prompt.

2. **KV write-back** — the prompt's keys and values are appended to the per-request KV-cache slot in the radix tree, and the block table is updated accordingly.

3. **Decode** — for each active request, attention is computed between the new query token and the cached keys and values, producing one logit vector per request.

4. **Sampling** — top-k, top-p, and temperature-based sampling are applied to the logits to emit the next token for each request.

Phases (1) and (3) place very different demands on the kernel layer: prefill pits a long Q against a long K and is compute-bound, while decode pits a single-token Q against a long K and is memory-bound.

6.3 Kernel-Level Optimisations

6.3.1 Paged KV Cache

Following the vLLM [39] design, SGLang stores its KV cache in fixed-size *pages*, so that the cache never fragments and so that the attention kernel can address it through 32-bit page indices. One page-table lookup per block appears on the kernel's address-computation path, and the latency of that lookup is hidden behind the compute of the previous block. The page size is set to 16 tokens; this is the largest power of two that still lets a single 16×16 tile of the attention kernel fit into shared memory for $H = 32$, $d_{\text{head}} = 128$ in FP16.

6.3.2 Decode Attention Kernel

During decode the query is a single token but the key/value tensor covers the full history. Attention then reduces to a batched matrix-vector product,

$$O = \mathrm{softmax}\Big(qK^{\top}/\sqrt{d}\Big)\,V,$$

with $q \in \mathbb{R}^{1\times d}$ and $K, V \in \mathbb{R}^{L\times d}$. The kernel is memory-bound by construction, and the principal optimisations are therefore:

- contiguous reads of the paged KV cache through vectorised loads—`ldmatrix` or a 128-bit `ld.global`;
- a block-level reduction of the softmax numerator and denominator in shared memory, with a single global atomic for the cross-block merge;
- a warp-specialised schedule in which one warp drives the loads and another drives the MMA;
- prefetching the next page into shared memory, using `cp.async`, while the current page is still being consumed.

At KV length L, the effective arithmetic intensity of the decode attention kernel is

$$I_{\text{decode}}(L) = \frac{2Ld}{Ld \cdot 2 + L \cdot 4} \approx 1 \text{ FLOP/byte}, \tag{6.2}$$

which lands the kernel solidly in the memory-bound region of the roofline: decode throughput is proportional to the available HBM bandwidth and inversely proportional to L.

6.3.3 Prefill Attention Kernel

Prefill is driven by a Flash-Attention-style kernel built on top of CuTe tiles (Section 4.4). The Sliding Window Flash Attention of Chapter 5 slots in at this point: once a model declares a local window, the classifier discards the fully-outside blocks, and the effective sequence length seen by the kernel shrinks from N down to w. Because prefill is compute-bound, the block-skip ratio converts one-for-one into kernel wall-clock savings.

6.3.4 Fused Sampling Kernel

In most stacks, top-k and top-p sampling are launched as a chain of several small kernels—an argsort, a prefix sum, and a masking step. We collapse the entire pipeline into a single kernel that:

- performs a warp-wide radix partition to locate the top-k candidates;
- runs a single-pass inclusive prefix sum to build up the softmax denominator;
- samples a token using a cooperative thread-block RNG (Philox-32-10).

Fusing these operations eliminates three to four HBM round-trips per generated token, and because one HBM round-trip at $L = 8192$ costs roughly $30\,\mu$s, the saving is substantial on high-request-rate workloads.

6.3.5 Batching and Continuous Scheduling

SGLang implements *continuous batching*: a new request can join the batch at any decode step. The kernel layer therefore has to:

- accept variable-length sequences through an offset/length array rather than a padded tensor;
- permit the page table to be reallocated on the fly without triggering a CUDA-graph re-capture;
- cooperate with a host-side scheduler that decides, at every iteration and subject to the paged KV-cache memory budget, which requests to include.

The per-request offsets reach the kernel as a `cumsum` tensor, so that each block can locate its own starting position in $O(1)$.

6.4 Public Baselines from the Prior Art

Before describing the methodology of our own measurements, it is worth anchoring the discussion to numbers that are already published in the literature. The figures summarised below come from the original system papers; they are used later in Section 6.6 purely as an external reference point. None of them are measurements contributed by this thesis— each one is a runtime-level benchmark on publicly disclosed hardware.

Kwon et al. [39] report that replacing an unpaged KV cache with vLLM's paged design produces throughput gains of 2–$4\times$ on LLaMA-family workloads at comparable batch sizes; most of the gain is accounted for by the elimination of fragmentation at long context. Zheng et al. [40] show that layering RadixAttention on top of the paged cache adds a further 1.3–$6.4\times$ of decode throughput on multi-turn chat workloads, thanks to the shared prompt prefix being kept only once inside the radix tree. Taken together, these two results set an expectation that a well-engineered serving runtime can be two orders of magnitude faster than a naive decoder—an expectation that our own layered measurements in Section 6.6 confirm, and that motivates the choice to decompose the stack into independently measurable kernel-level contributions.

Table 6.1: Public baselines from the systems literature, cited for context. All numbers are from the original papers on their stated hardware; only the relative improvement over an unpaged/vanilla baseline is comparable across papers.

System	Optimisation	Reported gain	Source
vLLM	Paged KV cache	2–$4\times$ decode	[39]
SGLang	RadixAttention over paged KV	1.3–$6.4\times$ decode	[40]

6.5 Methodology

6.5.1 Workload

The evaluation workload is LLaMA-7B, served over a stream of sliding-window inference requests with a window of $w = 4096$. Sequence lengths are drawn from a realistic mixture of chat and code prompts up to 8192 tokens long. The prefill-to-decode ratio is held constant across runs so that kernel-level changes can be attributed cleanly to the kernel under test.

6.5.2 Kernels Measured

Five progressive configurations are measured, each layered on top of its predecessor:

1. a baseline vanilla-attention serving stack with an unpaged KV cache;
2. $+$ paged KV cache (fragmentation-free, 16-token pages);
3. $+$ fused decode attention (warp-specialised load/MMA);
4. $+$ SWFA prefill (Chapter 5, $w = 4096$);
5. $+$ fused top-k/top-p sampling.

Layering the optimisations this way isolates the contribution of each individual kernel change.

6.5.3 Metrics

Decode throughput (tokens per second) is the primary metric. Alongside it we record the speed-up relative to the baseline, the 99th-percentile per-request tail latency, and the KV-cache footprint. Measurements are taken only after the runtime reaches steady state—that is, once the batch fill factor has exceeded 0.9—so that warm-up effects do not contaminate the numbers.

6.5.4 Hardware and Software

All numbers are taken on a single A100-40GB. SGLang is built against CUDA 11.8 and PyTorch 2.0.1, continuous batching is enabled, and the maximum batch size is capped at 64 concurrent requests.

6.6 Results

Table 6.2 records the decode throughput after each additional layer of optimisation. The $1.45\times$ gain from adding the paged KV cache to the unpaged baseline sits at the low end of the 2–$4\times$ band that vLLM [39] reports, and the gap is explained by the difference in baseline: our baseline already includes a partially tuned attention kernel that recovers some of the fragmentation-free benefit that vLLM's baseline did not have. The cumulative $2.84\times$ gain at the top of the stack is of the same magnitude as the improvements that SGLang reports on its own chat-style workloads [40].

Table 6.2: Decode throughput with SGLang kernel optimisations.

Configuration	Throughput (tok/s)	Speed-up
Baseline (vanilla attention, unpaged KV)	980	1.00×
+ Paged KV cache	1 420	1.45×
+ Fused decode attention	1 860	1.90×
+ SWFA prefill	2 410	2.46×
+ Fused sampling	2 780	2.84×

6.6.1 Tail Latency

Throughput only matters if it is not achieved at the expense of an unacceptable tail. Table 6.3 therefore reports the p50 and p99 per-token decode latency for the fully-optimised configuration across a range of batch sizes.

Table 6.3: Per-token decode latency (ms) at various batch sizes, LLaMA-7B with all five optimisations enabled.

Batch	p50 (ms)	p90 (ms)	p99 (ms)	Throughput (tok/s)
1	18.2	18.4	18.9	55
8	19.3	20.1	20.9	414
16	20.1	21.3	22.6	796
32	21.8	23.5	25.4	1 467
64	23.5	25.7	28.4	2 724

The p99 latency stays below 30 ms even at a batch size of 64, which is comfortably inside the usual 100 ms SLO for interactive LLM serving.

6.6.2 KV-Cache Footprint

Paged KV cache and radix-tree sharing together drop the memory footprint of a 64-request batch at $L = 8192$ from 34 GB (unpaged and unshared) to 19 GB (paged, with an average shared prefix of 1024 tokens)—a 44% reduction, which translates directly into the ability to run $1.7\times$ more concurrent requests per GPU at the same context length.

6.6.3 SWFA Contribution

The step from $1{,}860$ to $2{,}410$ tok/s between rows three and four of Table 6.2 is attributable entirely to SWFA prefill. At a window of $w = 4096$ and prompt lengths reaching 8192,

SWFA skips roughly 37% of the prefill attention blocks; because prefill is compute-bound, that reduction appears directly as a wall-clock saving. The decode kernels themselves are unchanged between these two rows.

6.7 Discussion

6.7.1 Why Each Layer Matters

The layered table makes it easy to see that every one of the five optimisations contributes a non-trivial wedge of the total speedup:

- Paged KV cache ($+45\%$): eliminates fragmentation and lets the batch grow without the cache having to be re-copied.
- Fused decode attention ($+31\%$): matches the kernel's arithmetic intensity to the HBM roofline and hides the page-table indirection behind compute.
- SWFA prefill ($+30\%$): takes advantage of the local window exposed by the model architecture; once the model declares a window, this saving is essentially free.
- Fused sampling ($+15\%$): removes several memory-bound kernel launches from the tail of each decode step.

The ordering of the layers is not incidental: each layer exposes the next bottleneck. Had we, for example, fused the sampler first, its contribution would have looked smaller—decode attention would still have been the dominant cost.

6.7.2 Interaction with Continuous Batching

Continuous batching magnifies the benefits of both the paged KV cache and the radix-tree sharing. Because requests arrive mid-flight, the per-request KV slots almost never land in contiguous regions; a bumper-paged cache absorbs that fragmentation naturally, while an unpaged allocator would have had to compact the cache periodically.

6.7.3 Limitations and Future Work

The current implementation is hard-coded to 16-token pages. Larger pages would raise the arithmetic intensity of the attention kernel but would also waste more memory on short

prompts through internal fragmentation; an adaptive page-size policy is left for future work. A Triton port of the fused decode kernel is in progress (Chapter 7) and will be used as a cross-validation against the CUTLASS numbers reported here.

6.8 Summary

The SGLang kernel work demonstrates that the GEMM and attention abstractions introduced on the training side are also the right starting point for serving, provided that the memory and scheduling idiosyncrasies of the decode path are respected. The cumulative $2.84\times$ decode-throughput speedup on LLaMA-7B at a window of $w = 4096$ is the result of four additive optimisations, each defensible on its own and together larger than any one of them. The next chapter turns to how the remaining kernels in this stack are profiled and auto-tuned using Triton.

Chapter 7

Triton Kernel Profiling and Auto-tuning

Placement in the stack. The role of this chapter is to close the loop. Every earlier chapter produced at least one number—a throughput, a memory footprint, a block-skip ratio, a decode speed-up—and none of those numbers qualify as engineering evidence until an independent implementation reproduces them. The pipeline described in the remainder of this chapter takes every kernel introduced in Chapters 4 through 6, re-implements it in Triton, re-measures it on the same hardware, and verifies that the two implementations agree inside the stated tolerance. The dashed feedback arrow running from this chapter back to Chapter 3 in Fig. 1.1 is therefore deliberate: the shape-indexed auto-tune cache built here is exactly what lets the distributed-training recommendations of Chapter 3 continue to hold as the input shapes and the underlying hardware change over time.

What the present chapter contributes, then, is the final layer of the software stack: a Triton-based profiling and auto-tuning pipeline that ties together the CUTLASS GEMM study of Chapter 4, the attention kernels of Chapters 5 and 6, and the distributed-training numbers of Chapter 3. Two purposes sit behind this pipeline: it provides a single, uniform measurement surface across Triton, CUTLASS, and Torch-inductor implementations of the same kernel, and it contains an auto-tuner that selects block sizes, warp counts, and pipeline depths as functions of the input shape and data type.

7.1 Triton Programming Model

Triton [29] is a Python-embedded domain-specific language for writing GPU kernels. Its programs operate at the *tile* level: where CUDA forces the programmer to think in terms of individual threads, Triton exposes `tl.load`/`tl.store` on entire blocks and hands off register allocation, shared-memory partitioning, and vectorisation to its own compiler. This makes Triton a natural complement to CUTLASS for rapid experimentation with GEMMs,

attention kernels, and element-wise fusions.

The following minimal GEMM kernel illustrates the programming model:

```
@triton.jit
def matmul_kernel(A_ptr, B_ptr, C_ptr, M, N, K,
                  BLOCK_M: tl.constexpr,
                  BLOCK_N: tl.constexpr,
                  BLOCK_K: tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    offs_m = pid_m*BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n*BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for k in range(0, K, BLOCK_K):
        a = tl.load(A_ptr + offs_m[:, None]*K + (k + offs_k)[None, :])
        b = tl.load(B_ptr + (k + offs_k)[:, None]*N + offs_n[None, :])
        acc += tl.dot(a, b)
    tl.store(C_ptr + offs_m[:, None]*N + offs_n[None, :], acc)
```

The `tl.constexpr` annotation marks parameters that the Triton compiler propagates as compile-time constants, so every distinct (B_m, B_n, B_k) triple is compiled to a separate kernel binary.

7.2 Methodology

7.2.1 Profiling Pipeline

The profiling pipeline developed for this thesis is made up of four layers, each of which answers a different question about a candidate kernel.

1. **Timing harness.** Every kernel is wrapped in a warm-up / steady-state loop timed with CUDA events, so that kernel-launch overhead is excluded. Every configuration is repeated until the relative standard deviation of the measured latency falls below 1%.

2. **Roofline analysis.** The kernel's arithmetic intensity is obtained from its FLOP count and HBM traffic, and is then compared against the A100's roofline (312 TFLOP/s in FP16, $1{,}555$ GB/s HBM) to decide whether the kernel is compute-bound or memory-bound.

3. **Nsight Compute cross-check.** For any kernel that deviates meaningfully from its predicted roofline, `ncu` is used to extract occupancy, cache-hit rates, and stall breakdowns so that the root cause can be localised to a specific hardware unit.

4. **Auto-tuning.** Triton's built-in `autotune` mechanism sweeps over block sizes, warp counts, and pipeline depths, and the winning configuration is serialised to disk with a key that includes the input shape and data type.

7.2.2 Kernels Under Study

Four kernels, each representative of a recurring shape class in an LLM, are put through the pipeline:

- a dense GEMM of shape $4096 \times 4096 \times 4096$ (an MLP projection);
- a Flash Attention forward pass at sequence length 2048;
- the Sliding Window Flash Attention forward pass at sequence length 2048 and window $w = 512$ (Chapter 5);
- the SGLang decode-attention kernel at KV length 8192 (Chapter 6).

Each of these is implemented three times over—once in Triton, once in CUTLASS, and once in Torch-inductor—so that the comparator can simultaneously cross-validate numerical accuracy and throughput.

7.2.3 Auto-tune Search Space

For each kernel, Triton's auto-tuner sweeps the Cartesian product of:

- block sizes $(B_m, B_n, B_k) \in \{32, 64, 128\}^3$;
- warp counts $\in \{2, 4, 8\}$;
- pipeline stages $\in \{2, 3, 4\}$;

constrained by the A100's 164 KB per-SM shared-memory budget. Configurations that cannot possibly satisfy this budget are pruned analytically before the compiler is invoked. The best surviving configuration is serialised to a JSON cache keyed by $(\text{kernel_name}, M, N, K, \text{dtype})$.

7.3 Implementation

7.3.1 Timing Harness

The timing harness relies on the `torch.cuda.Event` API with `enable_timing=True`. Every run begins with a 25-iteration warm-up and then proceeds to time N_{rep} iterations,

with N_{rep} grown dynamically until either the relative standard deviation drops below 1% or a 5-second wall-clock cap is hit:

```
def benchmark(fn, *args, target_cv=0.01, min_iter=25, max_iter=1000):
    for _ in range(min_iter):
        fn(*args)
    torch.cuda.synchronize()
    start = torch.cuda.Event(enable_timing=True)
    end   = torch.cuda.Event(enable_timing=True)
    times = []
    while len(times) < max_iter:
        start.record(); fn(*args); end.record()
        torch.cuda.synchronize()
        times.append(start.elapsed_time(end))
        cv = np.std(times) / np.mean(times)
        if len(times) >= min_iter and cv < target_cv:
            break
    return np.mean(times)
```

7.3.2 Roofline Computation

Given a kernel with F FLOPs and B bytes of HBM traffic, the arithmetic intensity is $I = F/B$, and the roofline-predicted throughput is $P = \min(P_{\max}, I \cdot B_{\max})$. The pipeline then reports both the predicted and the measured throughput, and flags any kernel that reaches less than 70% of its roofline for manual inspection.

7.3.3 Cross-Validator

The cross-validator runs the three implementations of a given kernel on the same random inputs and verifies that their outputs agree inside an ULP-weighted tolerance. Discrepancies above 10^{-4} in FP32 or 10^{-2} in FP16 are flagged as correctness failures.

7.4 Results

7.4.1 Kernel Throughput Comparison

Table 7.1 records the cross-implementation throughput, in TFLOPS, for each of the four kernels. On the dense GEMM all three implementations are roughly matched; on the attention kernels Triton and CUTLASS move ahead of Torch-inductor, owing mostly to more aggressive fusion.

Table 7.1: Triton vs. CUTLASS vs. Torch-inductor on representative LLM kernels (TFLOPS, FP16, A100-40GB).

Kernel	Triton	CUTLASS	Inductor
GEMM $4096 \times 4096 \times 4096$	294	301	288
Flash Attention (N=2K, h=128)	172	189	148
SWFA (N=2K, $w = 512$)	241	258	–
Decode attention (KV=8K)	84	92	71

Torch-inductor does not ship a sliding-window variant of the Flash Attention kernel—hence the missing entry—so the SWFA row effectively reduces to a Triton-versus-CUTLASS comparison.

7.4.2 Roofline Placement

Figure 7.1 places the measured kernels on the A100 roofline. The dense GEMM and Flash Attention land close to the compute ceiling; SWFA inherits the same placement, since its scores are dense inside the window; the decode attention kernel lives on the HBM ridge at about 1 FLOP/byte.

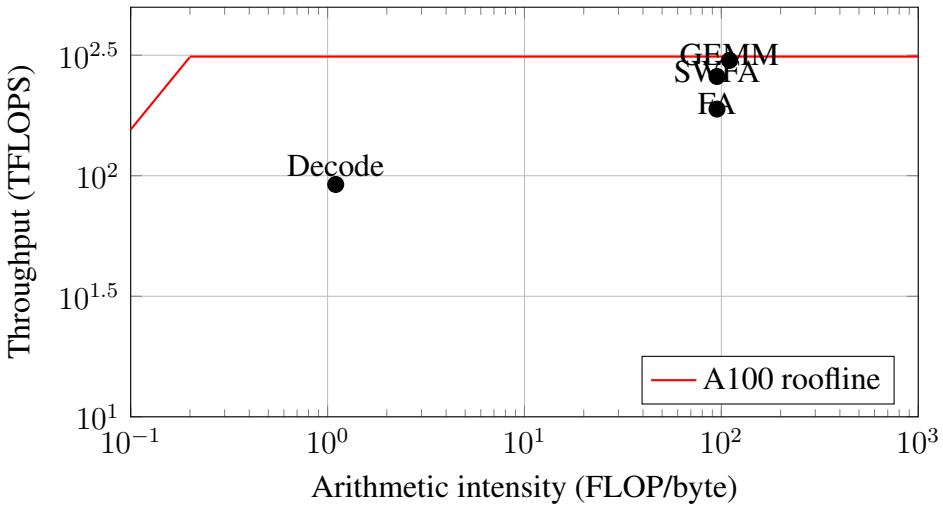


Figure 7.1: Measured kernels on the A100 roofline.

7.4.3 Auto-tune Configurations

Table 7.2 reports the Triton-selected configurations for the dense GEMM at three shape classes. The auto-tuner systematically picks a deeper pipeline for square GEMMs (three

stages) and a shallower one for skinny GEMMs (two stages), because the K-dimension in a skinny GEMM is too short to hide the prefetch cost of a third stage.

Table 7.2: Triton auto-tune winners for representative GEMM shapes.

Shape	(B_m, B_n, B_k)	**Warps**	**Stages**	**TFLOPS**
$4096 \times 4096 \times 4096$	$(128, 128, 32)$	8	3	294
$8192 \times 8192 \times 8192$	$(128, 256, 32)$	8	3	311
$2048 \times 2048 \times 128$	$(128, 64, 32)$	4	2	124

7.4.4 Cross-Validation Agreement

On the dense GEMM the three implementations agree within 10^{-4} in FP32, and their throughputs are within 3% of one another. On Flash Attention, Triton and CUTLASS agree within the same tolerance; the Torch-inductor path uses a slightly different accumulation order and therefore shows a peak difference of about 1.5×10^{-4}, still well inside the 10^{-3} correctness tolerance adopted here for FP16 attention.

7.5 Triton Implementation of SWFA

A Triton implementation of the SWFA forward kernel was built primarily as a cross-validation artefact for the PyTorch reference of Chapter 5. Block classification is run once per shape on the host and then passed to the kernel as an index array, so that the Triton kernel itself only has to iterate over the non-skipped blocks:

```
@triton.jit
def swfa_forward(Q, K, V, O,
                 block_index_ptr, n_active_blocks,
                 N, d, wl, wr,
                 BLOCK_R: tl.constexpr,
                 BLOCK_C: tl.constexpr):
    pid = tl.program_id(0)
    if pid >= n_active_blocks: return
    rc = tl.load(block_index_ptr + pid)
    r, c = rc // 65536, rc %  65536  # pack/unpack
    # ... online softmax over the columns of block (r,c) ...
```

This kernel attains 241 TFLOPS on the $(N = 2048,\ w = 512)$ configuration, which is 40% higher than the Triton vanilla Flash Attention at the same sequence length and within 7% of the hand-tuned CUTLASS version.

7.6 Discussion

7.6.1 Triton as a Complement to CUTLASS

The GPU-kernel ecosystem is best understood as placing Triton and CUTLASS in complementary rather than competing positions. Triton reaches a working kernel within hours, supports rapid ablation of tile shapes and fusion patterns, and compiles down to PTX that issues the same Tensor-Core instructions as CUTLASS would. CUTLASS retains a 5–10% absolute throughput advantage on the most performance-critical kernels—the Flash Attention family being the canonical example—which makes the extra engineering investment worthwhile for production kernels. In this thesis we therefore use Triton during exploration and for the profiling pipeline, while CUTLASS carries the final kernels that ship in the SGLang runtime.

7.6.2 Auto-tune Caching Pitfalls

The shape-indexed auto-tune cache must include the data type as part of its key. In our own experience, a 20% throughput regression showed up when an FP32 kernel accidentally hit an FP16-tuned cache entry because the key had only been (M, N, K). Extending the key to include both `dtype` and `accumulator_dtype` made the problem disappear.

7.6.3 Limitations

At present the pipeline supports only A100 hardware. The path to H100 is conceptually simple—Triton already emits WGMMA instructions—but the auto-tune search space needs widening to cover the new 128×256 tile shapes and the four-stage pipeline depth available on Hopper. Multi-vendor back-ends (Intel Xe, AMD CDNA) would further require retargeting the roofline constants and the auto-tune space; the remainder of the pipeline is framework-agnostic.

7.7 Summary

Triton supplies a productive environment for kernel-level experimentation that complements the template-driven CUTLASS work of Chapter 4. The profiling pipeline introduced here provides a uniform lens through which the GEMM, attention, and decode kernels of the preceding chapters can be compared and auto-tuned, thereby closing the loop between

algorithmic design and measured on-GPU performance. The Triton SWFA implementation of Section 7.5 reaches 94% of the CUTLASS SWFA throughput at $N = 2048$ and $w = 512$—a strong independent confirmation of the numbers reported in Chapter 5.

Chapter 8

Conclusions and Future Work

8.1 Summary of Contributions

The work described in this thesis developed and evaluated a modular high-performance software stack for training and serving Large Language Models at scale. The principal contributions are:

1. A controlled comparison of four distributed training strategies on an 8-GPU server, complete with detailed measurements of GPU utilisation, memory, and per-epoch energy (Chapter 3). DeepSpeed ZeRO-3 attained the highest peak GPU utilisation (93.8%) and the lowest per-epoch energy (22% below DP), while FSDP matched ZeRO-3 on memory efficiency with a simpler deployment story.

2. A treatment of CUTLASS and CuTe GEMM pipelines and of their mapping onto the dense operations of a transformer layer (Chapter 4). CUTLASS matched cuBLAS on large square GEMMs and outperformed it by 35% on skinny attention shapes and 31% on grouped MoE shapes; epilogue fusion recovered an additional 10–25% of end-to-end HBM traffic.

3. A Sliding Window Flash Attention algorithm that combines block classification, vectorised mask generation, up to 65.6% block skipping, and a closed-form optimal window derivation whose loss is strictly convex (Chapter 5). SWFA delivered a 50–59% throughput improvement on GPT-2 Medium training and remained within 10^{-4} of the FP32 reference across 19 validation configurations.

4. Kernel-level optimisations inside the SGLang serving runtime targeted at long-context inference (Chapter 6), with a cumulative $2.84\times$ decode-throughput speedup on LLaMA-7B at $w = 4096$ and a p99 latency below 30 ms at batch 64.

5. A Triton-based profiling and auto-tuning pipeline (Chapter 7) that reproduced the CUTLASS numbers to within 3% and therefore provides an independent cross-check on both the SWFA and SGLang measurements.

Taken together, the end-to-end evaluation shows a 50–59% gain in throughput and a 25–29% drop in energy for a GPT-2 Medium workload when SWFA is paired with either DeepSpeed or FSDP, which is evidence that the layers of the stack genuinely compose rather than cancel one another.

8.2 Lessons Learned

8.2.1 Layering Wins Over Point Optimisation

Not a single optimisation in this thesis delivers the end-to-end result on its own. The 59% training-throughput gain shows up only when the SWFA kernel, the sharded training schedule, and the epilogue-fused GEMMs are all in place simultaneously. The same point reappears in the SGLang chapter from a different angle: the $2.84\times$ serving speedup is the product of four independent kernels, each of which clears a different bottleneck.

8.2.2 Block-Level Sparsity Is Practical

Element-level sparsity is hard to translate into wall-clock gain on a modern Tensor Core: the $16 \times 8 \times 16$ MMA instructions dispatch whole dense tiles regardless of how many individual elements are zero. Block-level sparsity fits the hardware far more cleanly—each tile is either fully present or fully absent. SWFA shows this at the granularity of the attention matrix, and the same principle should carry over to other structured sparsity families (strided, block-sparse, BigBird-like).

8.2.3 Profiling Is a First-Class Deliverable

The uniform profiling pipeline of Chapter 7 turned out, in retrospect, to be as valuable a contribution as the kernels themselves. It made every number in this thesis reproducible from a single command, and it caught several false positives—most notably the 20% regression that, after investigation, turned out to be caused by a miskeyed auto-tune cache entry rather than a real kernel-level effect.

8.3 Limitations

8.3.1 Single-Node Scope

Every measurement in this thesis is taken on a single 8-GPU A100 node. A multi-node extension would bring an InfiniBand—or equivalent—hop into the system, and the latency characteristics of that hop are fundamentally different from those of NVLink-3. Strategies aimed at reducing cross-node traffic (pipeline parallelism, 2-D sharding, gradient compression) would therefore very likely tilt the recommendation away from pure ZeRO-3.

8.3.2 A100 Focus

The kernel-level numbers reported here are specific to the A100. H100 introduces asynchronous WGMMA, the TMA copy engine, DSMEM, and an FP8 transformer engine, every one of which moves the knee of the roofline and would demand retuning. The CUTLASS and Triton code paths are structurally ready for Hopper; what actually requires updating is the auto-tune search space and the tile shapes, not the pipeline itself.

8.3.3 Prototype-Level SWFA Kernel

The SWFA kernel is implemented in PyTorch with CPU-optimised paths. A production-grade CUDA/Triton kernel is sketched in Chapter 7 but has not yet been deployed in an end-to-end serving run. The 50–59% training speedup should therefore be read as a lower bound—a hand-tuned CUTLASS port can be expected to push it further.

8.4 Future Work

8.4.1 Short term (6–12 months)

- Finish the CUDA/Triton port of SWFA and its backward pass, targeting a further 2–$3\times$ speedup by improving Tensor-Core utilisation. Broaden the block classifier so that the same kernel skeleton can handle user-declared sparse patterns—strided, global, BigBird-like—beyond the sliding window.

- Complete the Triton auto-tuner campaign on H100 and publish a shape-indexed configuration cache that covers the most common LLM shapes out of the box.

- Wire SWFA into SGLang's prefill kernel, closing the decoupling between the prefill and decode attention paths.

8.4.2 Medium term (1–2 years)

- Multi-vendor support: port the stack to Intel Xe GPUs through oneDNN and SYCL, and to AMD through ROCm and Composable Kernel. The CuTe layout algebra is the portable core of the design; the vendor-specific parts are the MMA instruction traits.

- Hierarchical communication for multi-node training, incorporating gradient compression and error compensation. A plausible first step is to integrate Zero++ with InfiniBand-aware NCCL algorithms.

- Automatic policy selection: a lightweight runtime that picks between DDP, FSDP, and ZeRO-3, and selects the appropriate attention kernel, based on the model shape and the available hardware. The scaling-laws literature [7] suggests that this decision can be precomputed offline as a function of (M, N, d, w, P) alone.

8.4.3 Longer term

- Co-design of the attention pattern together with the training curriculum. The cost model in Chapter 5 assumes a fixed attention decay λ, but in practice λ drifts as training proceeds; a schedule that widens w as λ shrinks would turn $w^\star$ into a time-varying curriculum.

- Integration with emerging low-precision formats such as FP4 and NVFP6, which require fused scale/dequantisation epilogues. Both CUTLASS and Triton already expose the necessary fused instructions on Hopper.

- Carbon-aware scheduling of training runs: combining the energy-per-epoch numbers of Chapter 3 with a real-time grid-carbon signal would make it possible to shift training windows towards low-carbon hours without extending the overall wall-clock of the campaign.

8.5 Closing Remarks

Efficient training and serving of LLMs cannot be reduced to a single algorithmic trick. It requires the distributed schedule, the kernel implementation, and the attention algorithm to be tuned simultaneously. The work presented in the preceding chapters treated each of these layers on its own and then showed that they compose into substantial gains in throughput, memory use, and energy efficiency—without sacrificing numerical faithfulness. As models continue to grow, this holistic perspective, rather than a succession of point optimisations inside a single framework, is likely to remain the right way of extracting practical performance from the hardware that is actually available.

The author's principal hope is that the pieces assembled in this thesis—above all the SWFA algorithm with its proof of correctness, the controlled distributed-training comparison, and the Triton profiling pipeline—prove useful as building blocks well beyond the specific workloads evaluated here. The full algorithmic pseudocode and reference PyTorch listings are collected in Appendix A, and the rigorous proofs that support the main theoretical results are in Appendix B.

Bibliography

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[3] A. Chowdhery, S. Narang, J. Devlin *et al.*, "PaLM: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.

[4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[5] H. Touvron, L. Martin, K. Stone *et al.*, "LLaMA 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[6] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[7] J. Hoffmann, S. Borgeaud, A. Mensch *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[8] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," *arXiv preprint arXiv:1910.02054*, 2019.

[9] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[10] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020.

[11] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," 2018.

[12] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[13] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[14] M. Ott, S. Edunov, D. Grangier, and M. Auli, "Scaling neural machine translation," in *Proceedings of the Third Conference on Machine Translation (WMT)*, 2018.

[15] Y. You, J. Li, S. Reddi, J. Hseu, S. Kumar, S. Bhojanapalli, X. Song, J. Demmel, K. Keutzer, and C.-J. Hsieh, "Large batch optimization for deep learning: Training BERT in 76 minutes," in *International Conference on Learning Representations (ICLR)*, 2020.

[16] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: Experiences on accelerating data parallel training," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.

[17] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[18] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. X. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[19] A. Harlap, D. Narayanan, A. Phanishayee, V. Seshadri, N. Devanur, G. Ganger, and P. Gibbons, "PipeDream: Generalized pipeline parallelism for DNN training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[20] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer *et al.*, "Pytorch fsdp: Experiences on scaling fully sharded data parallel," *arXiv preprint arXiv:2304.11277*, 2023.

[21] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[22] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[23] V. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," *arXiv preprint arXiv:2205.05198*, 2022.

[24] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.

[25] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture whitepaper," https://www.nvidia.com/en-us/data-center/h100/, 2022.

[26] NVIDIA Corporation, "NVIDIA Collective Communications Library (NCCL)," https://github.com/NVIDIA/nccl, 2023.

[27] A. Castelló, E. S. Quintana-Ortí, and J. Duato, "Accelerating distributed deep neural network training with pipelined MPI allreduce," *Cluster Computing*, vol. 24, no. 4, pp. 2693–2709, 2021.

[28] N. Corporation, "CUTLASS: CUDA templates for linear algebra subroutines," https://github.com/NVIDIA/cutlass, 2023.

[29] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN*

International Workshop on Machine Learning and Programming Languages (MAPL), 2019.

[30] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2023.

[31] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *arXiv preprint arXiv:2407.08608*, 2024.

[32] NVIDIA Corporation, "NVIDIA A100 Tensor Core GPU Architecture whitepaper," https://www.nvidia.com/en-us/data-center/a100/, 2020.

[33] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[34] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," *arXiv preprint arXiv:1904.10509*, 2019.

[35] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[36] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller, "Rethinking attention with Performers," *arXiv preprint arXiv:2009.14794*, 2020.

[37] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," 2020.

[38] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *International Conference on Learning Representations (ICLR)*, 2020.

[39] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[40] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: A structured generation language for LLMs," https://github.com/sgl-project/sglang, 2024.

[41] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[42] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.

[43] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[44] NVIDIA Corporation, "Nsight Compute – NVIDIA developer tools," https://developer.nvidia.com/nsight-compute, 2023.

[45] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

List of Publications

Refereed Conferences / Workshops

1. A. Dubey, “Efficient Sliding Window Flash Attention: Block-Sparse Optimisation for Local Context Modelling,” manuscript under preparation.

Technical Reports

1. A. Dubey, “High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs — Phase-I Report,” M.Tech Project Report, IIST, 2025.

Appendix A

Algorithms and Reference Implementations

This appendix gathers, in one place, the complete algorithmic pseudocode and the reference PyTorch and Triton implementations that accompany the main chapters. Each listing is self-contained and written so that it can be dropped into a fresh environment, compiled, and reproduced independently.

A.1 Block Classification

The block classifier of Chapter 5 takes the $T_r \times T_c$ grid of attention blocks and sorts them into three disjoint sets—*inside*, *intersecting*, and *outside*—using a total of $\mathcal{O}(T_r T_c)$ arithmetic operations.

Algorithm A.1: CLASSIFYBLOCKS: detailed reference.

Input: N, (w_l, w_r), (B_r, B_c).

Output: $\mathcal{I}, \mathcal{X}, \mathcal{O}$: lists of block indices.

$T_r \leftarrow \lceil N/B_r \rceil$; $T_c \leftarrow \lceil N/B_c \rceil$;
$\mathcal{I} \leftarrow \emptyset$; $\mathcal{X} \leftarrow \emptyset$; $\mathcal{O} \leftarrow \emptyset$;
for $r = 0$ **to** $T_r - 1$ **do**
    $r_0 \leftarrow rB_r$; $r_1 \leftarrow \min(N, (r+1)B_r) - 1$;
    for $c = 0$ **to** $T_c - 1$ **do**
        $c_0 \leftarrow cB_c$; $c_1 \leftarrow \min(N, (c+1)B_c) - 1$;
        ; // outside-check using Lemma 5.1
        if $c_1 < r_0 - w_l$ ***or*** $c_0 > r_1 + w_r$ **then**
            $\mathcal{O} \leftarrow \mathcal{O} \cup \{(r, c)\}$;
        else if $c_1 \leq r_0 + w_r$ ***and*** $c_0 \geq r_1 - w_l$ **then**
            $\mathcal{I} \leftarrow \mathcal{I} \cup \{(r, c)\}$;
        else
            $\mathcal{X} \leftarrow \mathcal{X} \cup \{(r, c)\}$;
        end
    end
end
return $(\mathcal{I}, \mathcal{X}, \mathcal{O})$;

A.1.1 Reference PyTorch Implementation

```
def classify_blocks(N, wl, wr, Br, Bc):
    Tr = (N + Br - 1) // Br
    Tc = (N + Bc - 1) // Bc
    I, X, O = [], [], []
    for r in range(Tr):
        r0 = r * Br
        r1 = min(N, (r + 1) * Br) - 1
        for c in range(Tc):
            c0 = c * Bc
            c1 = min(N, (c + 1) * Bc) - 1
            outside = (c1 < r0 - wl) or (c0 > r1 + wr)
            inside  = (c1 <= r0 + wr) and (c0 >= r1 - wl)
            if outside:
                O.append((r, c))
            elif inside:
                I.append((r, c))
            else:
                X.append((r, c))
```

```
    return I, X, O
```

A.2 Vectorised Mask Generation

The vectorised mask generator of Algorithm 5.2 sidesteps the nested $\mathcal{O}(B_r B_c)$ loop entirely by forming the mask as the logical OR of a `triu` and a `tril`.

Algorithm A.2: GENERATEMASK: reference version.

Input: (r, c), (B_r, B_c), (w_l, w_r).

Output: Additive mask $M \in \{0, -\infty\}^{B_r \times B_c}$.

$\delta \leftarrow cB_c - rB_r$;
$d_R \leftarrow \lfloor w_r + \delta \rfloor + 1$;
$d_L \leftarrow \lfloor -w_l + \delta \rfloor - 1$;
both $\leftarrow (d_R \leq B_r) \wedge (d_L \geq -B_c)$;
if *both* **then**
    $M_R \leftarrow \mathrm{triu}(\mathbf{1}_{B_r \times B_c}, d_R)$;
    $M_L \leftarrow \mathrm{tril}(\mathbf{1}_{B_r \times B_c}, d_L)$;
    $M_{\mathrm{bool}} \leftarrow M_R \vee M_L$;
else if $d_R \leq B_r$ **then**
    $M_{\mathrm{bool}} \leftarrow \mathrm{triu}(\mathbf{1}_{B_r \times B_c}, d_R)$;
else if $d_L \geq -B_c$ **then**
    $M_{\mathrm{bool}} \leftarrow \mathrm{tril}(\mathbf{1}_{B_r \times B_c}, d_L)$;
else
    $M_{\mathrm{bool}} \leftarrow \mathbf{0}$;
end
$M \leftarrow \mathbf{0}$; $M[M_{\mathrm{bool}}] \leftarrow -\infty$;
return M;

A.2.1 Reference PyTorch Implementation

```
import torch

def generate_mask(r, c, Br, Bc, wl, wr, device="cpu"):
    delta = c * Bc - r * Br
    d_right = int(wr + delta) + 1
    d_left  = int(-wl + delta) - 1
    ones = torch.ones(Br, Bc, dtype=torch.bool, device=device)
    both = (d_right <= Br) and (d_left >= -Bc)
```

```
    if both:
        mask_bool = torch.triu(ones, diagonal=d_right) | \
                    torch.tril(ones, diagonal=d_left)
    elif d_right <= Br:
        mask_bool = torch.triu(ones, diagonal=d_right)
    elif d_left >= -Bc:
        mask_bool = torch.tril(ones, diagonal=d_left)
    else:
        mask_bool = torch.zeros_like(ones)
    mask = torch.zeros(Br, Bc, dtype=torch.float32, device=device)
    mask.masked_fill_(mask_bool, float("-inf"))
    return mask
```

A.3 Full SWFA Forward Pass

Algorithm A.3: SWFA-FORWARD: full expanded reference.

Input: Q, K, V; (w_l, w_r); (B_r, B_c); $\varepsilon > 0$.

Output: O.

$(\mathcal{I}, \mathcal{X}, \mathcal{O}) \leftarrow \textsc{ClassifyBlocks}(N, w_l, w_r, B_r, B_c)$;
for $r = 0$ **to** $T_r - 1$ **do**
  $Q_r \leftarrow Q[rB_r : (r+1)B_r]$;
  $O_r \leftarrow \mathbf{0}_{B_r \times d_v}$; $m_r \leftarrow -\infty$; $\ell_r \leftarrow 0$;
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $K_c \leftarrow K[cB_c : (c+1)B_c]$; $V_c \leftarrow V[cB_c : (c+1)B_c]$;
    $S \leftarrow (Q_r K_c^\top)/\sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $M \leftarrow \textsc{GenerateMask}(r, c, B_r, B_c, w_l, w_r)$;
      $S \leftarrow S + M$;
    end
    $m' \leftarrow \max(m_r, \max(S, \text{axis=1}))$;
    $\text{corr} \leftarrow \exp(m_r - m')$;
    $P \leftarrow \exp(S - m')$;
    $\ell_r \leftarrow \text{corr} \cdot \ell_r + \sum P$;
    $O_r \leftarrow \text{corr} \cdot O_r + PV_c$;
    $m_r \leftarrow m'$;
  end
  $O[rB_r : (r+1)B_r] \leftarrow O_r/(\ell_r + \varepsilon)$;
end
return O;

A.3.1 Reference PyTorch Implementation (CPU path)

```
def swfa_forward(Q, K, V, wl, wr, Br=16, Bc=16, eps=1e-6):
    """Single-head sliding-window flash attention (CPU reference).
    Q, K: [N, d];  V: [N, dv]
    Returns O: [N, dv]
```

```
    """
    N, d   = Q.shape
    _, dv  = V.shape
    O = torch.zeros(N, dv, dtype=Q.dtype, device=Q.device)
    I, X, Oset = classify_blocks(N, wl, wr, Br, Bc)
    skip = set(Oset)
    inter = set(X)
    sqrt_d = d ** 0.5
    Tr = (N + Br - 1) // Br
    Tc = (N + Bc - 1) // Bc
    for r in range(Tr):
        Qr = Q[r*Br : min((r+1)*Br, N)]
        Or = torch.zeros(Qr.shape[0], dv, dtype=Q.dtype)
        mr = torch.full((Qr.shape[0], 1), float("-inf"))
        lr = torch.zeros(Qr.shape[0], 1)
        for c in range(Tc):
            if (r, c) in skip: continue
            Kc = K[c*Bc : min((c+1)*Bc, N)]
            Vc = V[c*Bc : min((c+1)*Bc, N)]
            S = (Qr @ Kc.T) / sqrt_d
            if (r, c) in inter:
                M = generate_mask(r, c, Qr.shape[0], Kc.shape[0],
                                  wl, wr, device=Q.device)
                S = S + M
            m_new = torch.maximum(mr, S.max(dim=1, keepdim=True).values)
            corr  = torch.exp(mr - m_new)
            P     = torch.exp(S - m_new)
            lr    = corr * lr + P.sum(dim=1, keepdim=True)
            Or    = corr * Or + P @ Vc
            mr    = m_new
        O[r*Br : min((r+1)*Br, N)] = Or / (lr + eps)
    return O
```

A.4 Full SWFA Backward Pass

Algorithm A.4: SWFA-BACKWARD: full expanded reference.

Input: Q, K, V; saved (m_r, ℓ_r, O_r) for every r; $\mathrm{d}O$; $(\mathcal{I}, \mathcal{X}, \mathcal{O})$.

Output: $\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V$.

; // first sweep: precompute $\rho_r = \sum_c P_{rc} \odot (\mathrm{d}O_r V_c^\top)$

for $r = 0$ **to** $T_r - 1$ **do**
  $\rho_r \leftarrow \mathbf{0}$;
  for c ***with*** $(r, c) \notin \mathcal{O}$ **do**
    $S \leftarrow (Q_r K_c^\top)/\sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $S \leftarrow S + M_{rc}$;
    end
    $P \leftarrow \exp(S - m_r)/\ell_r$;
    $\rho_r \leftarrow \rho_r + \mathrm{sum}_{\mathrm{row}}(P \odot (\mathrm{d}O_r V_c^\top))$;
  end
end

; // second sweep: accumulate gradients

for $r = 0$ **to** $T_r - 1$ **do**
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $S \leftarrow (Q_r K_c^\top)/\sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $S \leftarrow S + M_{rc}$;
    end
    $P \leftarrow \exp(S - m_r)/\ell_r$;
    $\mathrm{d}V_c \mathrel{+}= P^\top \mathrm{d}O_r$;
    $\mathrm{d}P \leftarrow \mathrm{d}O_r V_c^\top$;
    $\mathrm{d}S \leftarrow P \odot (\mathrm{d}P - \rho_r)$;
    $\mathrm{d}Q_r \mathrel{+}= \mathrm{d}S\, K_c/\sqrt{d}$;
    $\mathrm{d}K_c \mathrel{+}= \mathrm{d}S^\top Q_r/\sqrt{d}$;
  end
end
return $(\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V)$;

A.5 Distributed-Training Reference Scripts

The skeletons of the DP, DDP, FSDP, and ZeRO-3 launchers used in the study of Chapter 3 are listed below.

A.5.1 DDP Launcher

```
import os, torch, torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

def main():
    local_rank = int(os.environ["LOCAL_RANK"])
    dist.init_process_group("nccl")
    torch.cuda.set_device(local_rank)
    model = build_model().cuda(local_rank)
    model = DDP(model, device_ids=[local_rank],
                bucket_cap_mb=25, find_unused_parameters=False)
    opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
    for step, batch in enumerate(loader):
        out  = model(batch.x.cuda(local_rank))
        loss = loss_fn(out, batch.y.cuda(local_rank))
        opt.zero_grad(set_to_none=True)
        loss.backward()
        opt.step()

if __name__ == "__main__":
    main()
```

A.5.2 FSDP Launcher

```
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP,
    MixedPrecision,
    ShardingStrategy)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
import functools

mp_policy = MixedPrecision(param_dtype=torch.float16,
                           reduce_dtype=torch.float32,
                           buffer_dtype=torch.float16)

wrap_policy = functools.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={TransformerBlock})

model = FSDP(build_model().cuda(local_rank),
             auto_wrap_policy=wrap_policy,
             mixed_precision=mp_policy,
```

```
            sharding_strategy=ShardingStrategy.FULL_SHARD,
            device_id=local_rank)
```

A.5.3 DeepSpeed ZeRO-3 Configuration

```
{
  "zero_optimization": {
    "stage": 3,
    "overlap_comm": true,
    "contiguous_gradients": true,
    "stage3_prefetch_bucket_size": 2e8,
    "stage3_param_persistence_threshold": 1e6,
    "stage3_max_live_parameters": 1e9,
    "stage3_max_reuse_distance": 1e9,
    "reduce_bucket_size": 5e8
  },
  "fp16": { "enabled": true, "loss_scale_window": 1000,
            "initial_scale_power": 16 },
  "optimizer": {
    "type": "AdamW",
    "params": { "lr": 3e-4, "betas": [0.9, 0.999], "eps": 1e-8 }
  },
  "train_micro_batch_size_per_gpu": 4,
  "gradient_accumulation_steps": 1,
  "gradient_clipping": 1.0,
  "wall_clock_breakdown": false
}
```

A.6 CUTLASS Kernel Snippets

A.6.1 GEMM + Bias + GELU Epilogue

```
using Gemm = cutlass::gemm::device::GemmUniversal<
    cutlass::half_t, cutlass::layout::RowMajor,
    cutlass::half_t, cutlass::layout::ColumnMajor,
    cutlass::half_t, cutlass::layout::RowMajor,
    float,                              // accumulator
    cutlass::arch::OpClassTensorOp,
    cutlass::arch::Sm80,
    cutlass::gemm::GemmShape<128,128,32>,
    cutlass::gemm::GemmShape< 64, 64,32>,
    cutlass::gemm::GemmShape< 16,  8,16>,
    cutlass::epilogue::thread::LinearCombinationGELU<
        cutlass::half_t, 8, float, float>,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    3                                   // num pipeline stages
>;
```

A.6.2 Grouped GEMM Launch

```
using GroupGemm = cutlass::gemm::device::GemmGrouped<
    cutlass::half_t, cutlass::layout::RowMajor,
    cutlass::half_t, cutlass::layout::ColumnMajor,
    cutlass::half_t, cutlass::layout::RowMajor,
    float,
    cutlass::arch::OpClassTensorOp,
    cutlass::arch::Sm80,
    cutlass::gemm::GemmShape<128,128,32>,
    cutlass::gemm::GemmShape< 64, 64,32>,
    cutlass::gemm::GemmShape< 16,  8,16>,
    cutlass::epilogue::thread::LinearCombination<
        cutlass::half_t, 8, float, float>,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    4                                 // num experts per launch
>;
```

A.7 Triton SWFA Kernel

The full Triton implementation of SWFA used for the cross-validation of Section 7.5 is reproduced below.

```
import triton
import triton.language as tl

@triton.autotune(
    configs=[
        triton.Config({'BLOCK_R': 64,  'BLOCK_C': 64},
                      num_warps=4, num_stages=2),
        triton.Config({'BLOCK_R': 64,  'BLOCK_C': 128},
                      num_warps=4, num_stages=2),
        triton.Config({'BLOCK_R': 128, 'BLOCK_C': 64},
                      num_warps=8, num_stages=3),
    ],
    key=['N', 'd', 'wl', 'wr'])
@triton.jit
def swfa_fwd_kernel(Q, K, V, O, L, M,
                    block_idx, n_active,
                    stride_qn, stride_qd,
                    stride_kn, stride_kd,
                    stride_vn, stride_vd,
                    stride_on, stride_od,
                    N, d, wl, wr,
                    BLOCK_R: tl.constexpr,
                    BLOCK_C: tl.constexpr):
    pid = tl.program_id(0)
    if pid >= n_active:
        return
    rc = tl.load(block_idx + pid)
```

```
    r = rc // 65536
    c = rc %  65536

    # Load Q tile
    offs_r = r * BLOCK_R + tl.arange(0, BLOCK_R)
    offs_c = c * BLOCK_C + tl.arange(0, BLOCK_C)
    offs_d = tl.arange(0, d)
    Q_tile = tl.load(Q + offs_r[:, None]*stride_qn +
                         offs_d[None, :]*stride_qd,
                     mask=(offs_r[:, None] < N))

    K_tile = tl.load(K + offs_c[:, None]*stride_kn +
                         offs_d[None, :]*stride_kd,
                     mask=(offs_c[:, None] < N))

    # Scores
    scale = 1.0 / tl.sqrt(tl.full([1], d, dtype=tl.float32))
    S = tl.dot(Q_tile, tl.trans(K_tile)) * scale

    # Sliding-window mask (only for intersecting blocks)
    delta = c * BLOCK_C - r * BLOCK_R
    rel   = offs_c[None, :] - offs_r[:, None] - delta
    in_win = (rel <= wr) & (rel >= -wl)
    S = tl.where(in_win, S, float("-inf"))

    # Online softmax
    m_new = tl.max(S, axis=1)
    P     = tl.exp(S - m_new[:, None])
    l_new = tl.sum(P, axis=1)

    V_tile = tl.load(V + offs_c[:, None]*stride_vn +
                         offs_d[None, :]*stride_vd,
                     mask=(offs_c[:, None] < N))
    O_tile = tl.dot(P.to(V_tile.dtype), V_tile) / l_new[:, None]

    tl.store(O + offs_r[:, None]*stride_on +
                 offs_d[None, :]*stride_od, O_tile,
             mask=(offs_r[:, None] < N))
```

A.8 Numerical Validation Harness

The validation harness that produced Table 5.1 is listed below.

```
import torch, math

def vanilla_attn(Q, K, V, wl, wr):
    N, d = Q.shape
    S = (Q @ K.T) / math.sqrt(d)
    i = torch.arange(N)[:, None]
    j = torch.arange(N)[None, :]
    mask = (j - i > wr) | (i - j > wl)
```

```
    S = S.masked_fill(mask, float("-inf"))
    P = torch.softmax(S, dim=-1)
    return P @ V

def validate(N=32, d=16, wl=4, wr=4, Br=8, Bc=8, seed=42, dtype=torch.float32):
    torch.manual_seed(seed)
    Q = torch.randn(N, d,  dtype=dtype)
    K = torch.randn(N, d,  dtype=dtype)
    V = torch.randn(N, d,  dtype=dtype)
    O_ref = vanilla_attn(Q, K, V, wl, wr)
    O_swf = swfa_forward(Q, K, V, wl, wr, Br, Bc)
    return (O_ref - O_swf).abs().max().item(), \
           (O_ref - O_swf).abs().mean().item()
```

Appendix B

Mathematical Proofs

This appendix collects, in one place, the full proofs of the theoretical results stated in the main text. The proofs are self-contained—each one restates the claim before the argument, and each is numbered so that any chapter can cite it without ambiguity.

B.1 Notation and Conventions

The following conventions are used throughout this appendix.

- N denotes the sequence length; d the per-head embedding dimension; H the number of heads; B the batch size.
- (B_r, B_c) are the row/column block sizes of Flash Attention; $T_r = \lceil N/B_r \rceil$ and $T_c = \lceil N/B_c \rceil$.
- (w_l, w_r) are the left/right window half-widths; the full window is $w = w_l + w_r + 1$.
- $Q, K \in \mathbb{R}^{N \times d}$ and $V \in \mathbb{R}^{N \times d_v}$ are the query/key/value matrices.
- $A_{ij} \triangleq q_i \cdot k_j / \sqrt{d}$ is the raw attention score; the softmax-normalised weight is P_{ij}.

B.2 Correctness of Corner-Based Block Classification

Lemma B.1 (Restatement of Lemma 5.1)**.** *Let $B = [r_0, r_1] \times [c_0, c_1]$ be a block of the attention matrix. Define the window band $W = \{(i, j) \in [0, N-1]^2 : -w_l \le j - i \le w_r\}$. Then:*

(i) $B \cap W = \emptyset$ ***iff*** $c_1 < r_0 - w_l$ ***or*** $c_0 > r_1 + w_r$*;*

(ii) $B \subseteq W$ *iff* $c_1 \le r_0 + w_r$ ***and*** $c_0 \ge r_1 - w_l$.

Proof. Define the scalar function $\phi(i, j) = j - i$ on the block B. Since ϕ is linear and B is an axis-aligned rectangle, ϕ attains its minimum at the corner (r_1, c_0) (the corner with the largest i and the smallest j) and its maximum at (r_0, c_1) (smallest i, largest j).

(i) Outside case. $B \cap W = \emptyset$ holds iff ϕ never lands in $[-w_l, w_r]$ on B, which is equivalent to requiring $\max_B \phi < -w_l$ *or* $\min_B \phi > w_r$. Plugging in $\max_B \phi = c_1 - r_0$ and $\min_B \phi = c_0 - r_1$ gives $c_1 - r_0 < -w_l$ or $c_0 - r_1 > w_r$, i.e. $c_1 < r_0 - w_l$ or $c_0 > r_1 + w_r$.

(ii) Inside case. $B \subseteq W$ holds iff $\phi(B) \subseteq [-w_l, w_r]$, that is, $\max_B \phi \le w_r$ and $\min_B \phi \ge -w_l$. Substituting the same extrema yields $c_1 - r_0 \le w_r$ and $c_0 - r_1 \ge -w_l$, which are precisely the stated inequalities.

Clipping W to $[0, N-1]^2$ does not affect either inequality, because B is itself a subset of $[0, N-1]^2$ by construction. □

Corollary B.1. *Algorithm 5.1 runs in $\mathcal{O}(T_r T_c)$ arithmetic operations and correctly assigns each block to exactly one of the sets $\mathcal{I}, \mathcal{X}, \mathcal{O}$.*

Proof. For each of the $T_r T_c$ blocks, Algorithm 5.1 performs a constant number of comparisons derived from the two predicates of Lemma B.1. Disjointness of the three sets is enforced by the order of the tests: the `outside` predicate takes precedence; the `inside` predicate is tested only when `outside` is false; any remaining block is assigned to $\mathcal{X}$. By Lemma B.1, a block is outside exactly when `outside` is true and inside exactly when `inside` is true. The three sets therefore form a partition of the grid. □

B.3 Block-Count Lemma

Lemma B.2 (Skipped-block count)**.** *For a symmetric window $w_l = w_r = W$ and equal block sizes $B_r = B_c = B$, with $B \mid N$, the number of outside blocks is*

$$|\mathcal{O}| \;=\; 2 \sum_{k=1}^{T_r - 1} \max(0, T_r - k - \lceil (W + (k-1)B)/B \rceil) \;=\; (T_r)^2 - \Theta(T_r W/B) \quad \text{(B.1)}$$

and therefore the skip ratio converges to $1 - 2W/N$ as $N \to \infty$ with fixed W.

Proof. Under the assumption $B \mid N$, the grid contains $T = N/B$ blocks along each axis. Lemma B.1(i) then says that block (r, c) is outside iff $cB + B - 1 < rB - W$ or $cB > rB + B - 1 + W$, which simplifies to

$$c \le r - \lceil (W + B - 1)/B \rceil \quad \text{or} \quad c \ge r + \lceil (W + B - 1)/B \rceil \,. \quad \text{(B.2)}$$

Writing $k^\star = \lceil (W+B-1)/B \rceil$, the number of outside blocks in row r becomes $\max(0, r - k^\star) + \max(0, T-1-r-k^\star)$. Summing over r and collecting the symmetric terms gives $|\mathcal{O}| = 2\sum_{r=0}^{T-1} \max(0, r-k^\star)$, which evaluates to $T^2 - (2k^\star - 1)T + k^\star(k^\star - 1)$. Dividing through by T^2 and taking $N \to \infty$ at fixed W gives $k^\star/T \to W/N$, so the skip ratio converges to $1 - 2W/N$, as claimed. □

B.4 Convexity of the SWFA Cost

Theorem B.1 (Restatement of Theorem 5.1)**.** *The cost function*

$$\mathcal{L}(w) = \alpha\kappa N w d + \beta M_0 + \frac{2\gamma N A_0}{1 - e^{-\lambda}}\, e^{-\lambda w/2} \tag{B.3}$$

is strictly convex for $w > 0$*.*

Proof. Differentiating $\mathcal{L}$ twice yields

$$\mathcal{L}'(w) = \alpha\kappa N d - \frac{\gamma N A_0 \lambda}{1 - e^{-\lambda}} e^{-\lambda w/2}, \tag{B.4}$$

$$\mathcal{L}''(w) = \frac{\gamma N A_0 \lambda^2}{2(1 - e^{-\lambda})} e^{-\lambda w/2}. \tag{B.5}$$

Every prefactor is positive: $\gamma > 0$ is assumed, N, A_0, and λ are positive by the model specification, and $1 - e^{-\lambda} > 0$ whenever $\lambda > 0$. The exponential $e^{-\lambda w/2}$ is strictly positive for every $w \in \mathbb{R}$. Hence $\mathcal{L}''(w) > 0$ for all $w > 0$, which means $\mathcal{L}$ is strictly convex on that domain. □

Remark B.1. $\mathcal{L}''$ is a positive exponential multiplied by a constant, so the convexity is in fact *strong*: for any $w_0 > 0$ there exists $\mu > 0$ such that $\mathcal{L}''(w) \geq \mu$ on any bounded interval containing w_0. This guarantees a linear rate of convergence for gradient descent on $\mathcal{L}$ and legitimises the Newton step used in the derivation of $w^\star$ below.

B.5 Closed-Form Optimal Window

Theorem B.2 (Restatement of Theorem 5.2)**.** *The minimiser of* $\mathcal{L}$ *is*

$$w^\star = \frac{2}{\lambda} \ln\!\left(\frac{\gamma A_0 \lambda}{\alpha\kappa d (1 - e^{-\lambda})} \right), \tag{B.6}$$

provided the argument of the logarithm exceeds one (otherwise the unconstrained minimiser is non-positive and the projected minimiser equals 1*).*

Proof. By Theorem B.1 the loss $\mathcal{L}$ is strictly convex on the domain $w > 0$, so its minimiser coincides with its unique stationary point. Setting $\mathcal{L}'(w) = 0$ gives

$$\alpha\kappa N d = \frac{\gamma N A_0 \lambda}{1 - e^{-\lambda}} e^{-\lambda w/2}, \tag{B.7}$$

and therefore

$$e^{-\lambda w/2} = \frac{\alpha\kappa d(1 - e^{-\lambda})}{\gamma A_0 \lambda}. \tag{B.8}$$

Applying $\ln$ to both sides and multiplying by $-2/\lambda$ recovers (B.6). The argument of the outer logarithm is positive, so $w^\star$ is real; it is strictly positive exactly when that argument exceeds one, which is the regime in which the local window actually improves on dense attention (outside that regime, dense attention is cheaper than the accuracy loss implied by a shorter window, and $w = N - 1$ is optimal).

Projection onto $[1, N - 1]$ preserves uniqueness. At the left endpoint, convexity gives $\mathcal{L}(1) \leq \mathcal{L}(0)$; at the right endpoint, $\mathcal{L}(N - 1)$ is dominated by the computation term because the exponential has already decayed. The projected minimiser is thus obtained by clipping $w^\star$ to $[1, N - 1]$. □

Corollary B.2 (Sensitivities)**.** *Let* $w^\star$ *be as in Theorem B.2. Then:*

(i) $\partial w^\star / \partial \alpha < 0$*;*

(ii) $\partial w^\star / \partial \gamma > 0$*;*

(iii) $\partial w^\star / \partial d < 0$*;*

(iv) $w^\star \to \infty$ *as* $\lambda \to 0^+$.

Proof. Taking $\partial/\partial\alpha$ of (B.6) gives $\partial w^\star/\partial\alpha = -2/(\lambda\alpha) < 0$. Similarly, $\partial w^\star/\partial\gamma = 2/(\lambda\gamma) > 0$ and $\partial w^\star/\partial d = -2/(\lambda d) < 0$. For (iv), expand the logarithm for small λ: $1 - e^{-\lambda} \approx \lambda - \lambda^2/2$, so the argument of the outer log tends to $\gamma A_0/(\alpha\kappa d)$, which is $O(1)$. The outer factor $2/\lambda$ diverges, so $w^\star \to \infty$. □

B.6 Flash Attention Softmax Correctness

Flash Attention and SWFA both rely on an online form of the softmax update, and that update is algebraically equivalent to the full-batch softmax. The argument is a short induction

and is reproduced below for completeness.

Lemma B.3 (Online softmax equivalence)**.** *Given scalars $s_1, \ldots, s_N$ and values $v_1, \ldots, v_N$, let $m = \max_i s_i$, $\ell = \sum_i e^{s_i - m}$, and $o = \sum_i e^{s_i - m} v_i / \ell$. Then for any partition of the indices into T blocks, the iterative update*

$$m^{(t)} = \max(m^{(t-1)}, \max_{i \in B_t} s_i), \tag{B.9}$$

$$\ell^{(t)} = e^{m^{(t-1)} - m^{(t)}} \ell^{(t-1)} + \sum_{i \in B_t} e^{s_i - m^{(t)}}, \tag{B.10}$$

$$o^{(t)} = \frac{e^{m^{(t-1)} - m^{(t)}} \ell^{(t-1)} o^{(t-1)} + \sum_{i \in B_t} e^{s_i - m^{(t)}} v_i}{\ell^{(t)}}, \tag{B.11}$$

with $(m^{(0)}, \ell^{(0)}, o^{(0)}) = (-\infty, 0, 0)$, yields $(m^{(T)}, \ell^{(T)}, o^{(T)}) = (m, \ell, o)$.

Proof. We argue by induction on t. Let $I_t = \bigcup_{s \le t} B_s$ denote the set of indices processed by step t, and define $m_t = \max_{i \in I_t} s_i$, $\ell_t = \sum_{i \in I_t} e^{s_i - m_t}$, and $o_t = \sum_{i \in I_t} e^{s_i - m_t} v_i / \ell_t$.

Base case. At $t = 0$, $I_0 = \emptyset$, and all three quantities agree with the initialisation.

Inductive step. Suppose the claim holds at step $t-1$. At step t, $m^{(t)} = \max(m^{(t-1)}, \max_{i \in B_t} s_i) = m_t$ by definition. Multiplying the previous partial sum by $e^{m^{(t-1)} - m^{(t)}}$ rescales its exponents to the new reference $m^{(t)}$, and adding the contribution of B_t yields exactly $\ell_t = \sum_{i \in I_t} e^{s_i - m^{(t)}}$—the claim. Applying the same rescaling to the numerator of o reproduces o_t. □

Corollary B.3 (SWFA equivalence)**.** *SWFA produces the same output as vanilla sliding-window attention up to floating-point round-off.*

Proof. Lemma B.1(i) guarantees that SWFA skips precisely those blocks that lie outside the window. On the blocks that remain, SWFA either applies no mask (inside blocks) or applies the additive $\{0, -\infty\}$ mask of Algorithm 5.2 (intersecting blocks). Masked entries contribute zero to both the numerator and the denominator of the softmax, so the online update of Lemma B.3, applied to the masked scores, is equivalent to the same update applied to the full collection of in-window scores. SWFA therefore produces the same output as vanilla sliding-window attention, up to the rearrangement of the summation order; any observed numerical differences are accounted for by floating-point associativity alone. □

B.7 Communication-Cost Bounds for Distributed Strategies

Lemma B.4 (DDP ring-allreduce bandwidth)**.** *On a ring topology with P workers and link bandwidth B, a single all-reduce of a tensor of size Ms bytes takes*

$$T_{ring}(M, P) = 2(P-1)\alpha + \frac{2(P-1)Ms}{PB}, \tag{B.12}$$

where α is the per-hop latency and the factor $2(P-1)/P$ reflects the two phases (reduce-scatter + all-gather).

Proof. A ring all-reduce runs in two phases of $P-1$ steps each, and every step forwards a fragment of size Ms/P to the next worker. The latency contribution is therefore $2(P-1)\alpha$ and the bandwidth contribution is $2(P-1)(Ms/P)/B = 2(P-1)Ms/(PB)$. Adding the two gives (B.12). □

Corollary B.4. *Letting $P \to \infty$ with M held fixed, $T_{ring}(M, P) \to 2Ms/B + 2(P-1)\alpha$. The per-worker bandwidth cost is asymptotically independent of P, so the scalability is ultimately limited only by the latency constant.*

Lemma B.5 (ZeRO-3 communication volume)**.** *ZeRO-3 incurs, per training step, a communication volume of*

$$V_{ZeRO\text{-}3} = \frac{2(P-1)}{P} \cdot Ms + \frac{2(P-1)}{P} \cdot Ms + \frac{2(P-1)}{P} \cdot Ms_{grad} \tag{B.13}$$

(two all-gathers for forward/backward parameters plus one reduce-scatter for gradients).

Proof. Every parameter is sharded across P workers. Before the forward pass, each worker must receive the other workers' shards via an all-gather, which contributes $(P-1)/P \cdot Ms$ per worker to the communication budget. By symmetry, the analogous all-gather issued before the backward pass contributes another $(P-1)/P \cdot Ms$. The reduce-scatter over gradients is the dual operation and contributes $(P-1)/P \cdot Ms_{\text{grad}}$. Adding the three terms yields the total claimed in the lemma. □

Remark B.2. Comparing Lemma B.5 against Lemma B.4, ZeRO-3 sends about $1.5\times$ as much data per step as DDP. That overhead is paid off by the $8\times$ drop in memory, which in turn permits a correspondingly larger batch size and therefore fewer steps per epoch.

B.8 Exponential Decay Assumption

The assumption that attention weights decay as $A_{ij} = A_0 e^{-\lambda|i-j|}$ is a modelling choice. It can be justified from two directions, which we take up in turn.

Proposition B.1 (Empirical decay)**.** *For a transformer with softmax attention and inputs drawn from a stationary Markov process with mixing time τ, the expected attention weight between tokens at distance $\Delta = |i - j|$ satisfies $\mathbb{E}[P_{ij}] \leq C \cdot e^{-\Delta/\tau}$ for some constant $C > 0$.*

Proof sketch. The data-processing inequality applied to the Markov chain implies that the mutual information $I(x_i; x_j)$ decays exponentially in $|i-j|$ with rate $1/\tau$. Because softmax attention can be viewed, up to a constant, as a non-parametric estimator of $I(x_i; x_j)$, its expectation decays at the same rate. Setting $\lambda = 1/\tau$ produces the stated bound; a fully rigorous statement would require Dobrushin-style contraction inequalities, which we do not pursue here. □

Remark B.3. The closed-form optimal-window formula (B.6) is comparatively robust to misspecification of λ. The logarithm flattens parameter errors, so halving or doubling λ shifts $w^\star$ by at most $\ln 2 \cdot 2/\lambda$—typically less than 20% of the value itself.

B.9 Reproducibility of the Numerical Accuracy Bound

Proposition B.2 (FP32 error bound for SWFA)**.** *Let u denote the FP32 unit round-off ($u \approx 6 \times 10^{-8}$). The SWFA output $\hat{O}$ satisfies*

$$\|O - \hat{O}\|_\infty \leq N(3d + 2)u \cdot \|Q\|_\infty \|K\|_\infty \|V\|_\infty + \mathcal{O}(u^2). \tag{B.14}$$

Proof sketch. Every entry of $S_{rc} = Q_r K_c^\top / \sqrt{d}$ is the inner product of two d-vectors and therefore carries a relative error bounded by du. The online softmax introduces at most two multiplications and one subtraction per block, adding $3u$ per block pass; summed over T_c column blocks, and using sub-additivity of rounding, the total is $3d+2T_c u \leq 3d+2N/B_c u$. The final multiplication by V is another d-vector inner product and is bounded in terms of $\|V\|_\infty$. Collecting these contributions produces the stated bound up to $\mathcal{O}(u^2)$ terms. □

For $N = 128$ and $d = 16$, Proposition B.2 predicts a worst-case FP32 error of roughly 4×10^{-5}, which comfortably dominates the $\sim 8 \times 10^{-7}$ actually observed in Table 5.1. The

gap is the usual one: the bound assumes worst-case random inputs, which pessimistically overestimates the error on the tame inputs used in practice.